

Part 1: Multimedia Compression — বেসিক ধারণা

1. Multimedia System

Start from the big picture before learning compression.

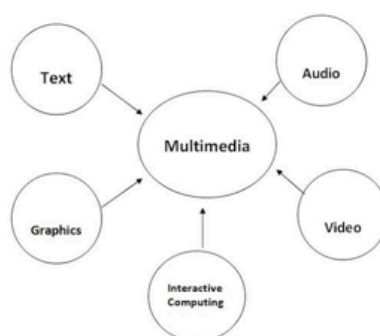
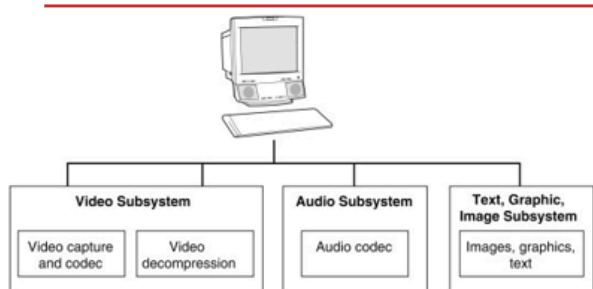


Fig: Various Element Of Multimedia

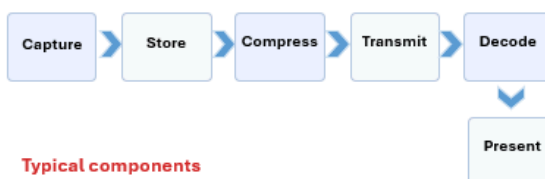
What is a Multimedia System?

Definition

A multimedia system is a computer-based system that stores, processes, transmits and presents multiple media types together: text, graphics, image, audio, video and animation.

Why it matters in computer graphics

Graphics is only one component. In a real application, a scene may include image, sound, video clips, subtitles, timing control and user interaction.



Typical components

- Input devices: camera, microphone, scanner
- CPU/GPU for processing
- Storage: SSD/HDD/cloud
- Network for delivery
- Display and speakers for output

১. Multimedia System কী?

সংজ্ঞা: Multimedia System হলো একটা কম্পিউটার-ভিত্তিক সিস্টেম (**computer-based system**), যেটা একাধিক ধরনের মাধ্যম (multiple types of media) — যেমন **text** (লেখা), **graphics** (গ্রাফিক্স), **image** (ছবি), **audio** (শব্দ), **video** (ভিডিও) — এগুলো **capture** (ধারণা), **store** (সংরক্ষণ), **compress** (সংকোচন), **transmit** (প্রেরণ) এবং **decode/present** (উপস্থাপন) করে।

গুরুত্বপূর্ণ বৈশিষ্ট্য:

- একাধিক মিডিয়া টাইপ একসাথে ব্যবহার হয় (integration)
- **Time-dependent data** (সময়-নির্ভর তথ্য) — audio, video ঠিক সময়ে চালাতে হয় (synchronization জরুরি — ঠেঁট নাড়া আর শব্দ মিলতে হবে)

- বিশাল ডেটা ভলিউম (**large data volume**) — raw ছবি/অডিও/ভিডিও অনেক জায়গা নেয়

Characteristics of a Multimedia System

Technical characteristics

- Integration of several media types in one application.
- Time-dependent data: audio and video must play at the correct rate.
- Large data volume: raw image, audio and video consume significant storage.
- Synchronization is essential: lip movement and sound should match.
- Interactive control: pause, rewind, zoom, seek, click, drag.

Real-life examples

- YouTube: video + audio + subtitles + thumbnails + controls.
- Online class: slides + camera feed + microphone + screen share.
- Game: 3D graphics + animation + background music + effects.
- Medical imaging system: image archive + annotations + video clips.
- Google Maps street view: image, text and voice navigation.

সহজ ভাষায় **Characteristics of a Multimedia System** (একটি মাল্টিমিডিয়া সিস্টেমের বৈশিষ্ট্যসমূহ) নিচে ভেঙে বুঝিয়ে দেওয়া হলো। স্লাইডটিতে দুটি অংশ রয়েছে: **Technical Characteristics** (কারিগরি বৈশিষ্ট্য) এবং **Real-life Examples** (বাস্তব জীবনের উদাহরণ)।

১. Technical Characteristics (টেকনিক্যাল বৈশিষ্ট্য)

- Integration of several media types:** একটি মাল্টিমিডিয়া সিস্টেমে কেবল লেখা (Text) বা কেবল ছবি (Image) থাকে না; বরং টেক্সট, অডিও, ভিডিও, অ্যানিমেশন ইত্যাদি একাধিক মিডিয়া একসাথে বা একটি একক অ্যাপ্লিকেশনে যুক্ত থাকে।
- Time-dependent data:** অডিও এবং ভিডিও নির্দিষ্ট সময় মেনে চলে। অর্থাৎ, ভিডিও বা গান যেন সঠিক স্পিড বা ফ্রেম রেটে (Correct rate) প্লে হয়, তা নিশ্চিত করা জরুরি; তা না হলে কথা আগে বা ভিডিও পেছনে পড়ে যাবে।
- Large data volume:** আনকম্প্রেসড বা র (Raw) ইমেজ, হাই-কোয়ালিটি অডিও এবং ভিডিও ফাইলগুলো প্রচুর মেমোরি এবং স্টোরেজ জায়গা দখল করে।
- Synchronization is essential:** লিপ-মিউভমেন্ট (Lip movement) বা কথা ও ঠোঁটের নড়াচড়ার মধ্যে মিল থাকা জরুরি। ভিডিওতে মানুষ কথা বলার সময়ের সাথে যদি অডিও না মেলে, তবে সেটা দেখার অভিজ্ঞতা নষ্ট হয়।
- Interactive control:** ব্যবহারকারী যেন সিস্টেমটিকে নিজের মতো নিয়ন্ত্রণ করতে পারে। যেমন—Pause, Rewind, Zoom, Click, Drag বা Seek করার সুবিধা থাকা।

২. Real-life Examples (বাস্তব জীবনের উদাহরণ)

- YouTube:** এখানে ভিডিও, সাউন্ড, সাবটাইটেল (Text), থাম্বনেইল (Image) এবং প্লে/পজ বাটন (Interactive control) একসাথে থাকে।
- Online Class (e.g., Zoom/Teams):** প্রজেন্টেশন স্লাইড, টিচারের ক্যামেরার ভিডিও, মাইক্রোফোনের ভয়েস এবং স্ক্রিন শেয়ারিংয়ের সমন্বয়ে কাজ করে।
- Games:** থ্রিডি গ্রাফিক্স, ক্যারেক্টার অ্যানিমেশন, ব্যাকগ্রাউন্ড মিউজিক এবং বিভিন্ন সাইন্ড ইফেক্টস একসাথে যুক্ত থাকে।

- **Medical Imaging System:** এক্স-রে বা সিটি স্ক্যানের ছবি, পেশেন্টের ডাটা এবং বিভিন্ন ভিডিও ক্লিপস একত্রিত থাকে।
- **Google Maps Street View:** রাস্তার ৩৬০ ডিগ্রি ছবি, লোকেশনের নাম (Text) এবং ভয়েস নেভিগেশন একসাথে কাজ করে।

সহজ কথায়, একাধিক মিডিয়ায় মেলবন্ধন, সঠিক সময়ের সিঙ্ক্রোনাইজেশন এবং ইউজারের কন্ট্রোলিং ক্ষমতাই হলো একটি সফল মাল্টিমিডিয়া সিস্টেমের মূল ভিত্তি।

Why Compression is Necessary in Multimedia

Raw data grows very fast

Media item	Simple raw size estimate	Meaning
1024 × 1024 grayscale image	1024×1024×8 bits = 8,388,608 bits ≈ 1 MB	One still image
1920 × 1080 RGB frame	1920×1080×24 bits ≈ 49,766,400 bits ≈ 5.93 MB	One full-color frame
30 fps Full HD video	5.93 MB × 30 ≈ 178 MB/s	Much too large for ordinary storage/streaming
CD-quality audio (stereo)	44,100 samples/s × 16 bits × 2 ≈ 1.41 Mb/s	Large for long recordings

Main reason

Compression reduces storage, transmission time, bandwidth and cost while trying to keep useful information.

Quick check

If one raw 4 MB image must be sent over a slow network, what happens if you compress it to 0.5 MB? Answer: less bandwidth, faster transfer, lower storage.

২. কেন **Compression (সংকোচন)** দরকার?

এই স্লাইডটিতে মাল্টিমিডিয়ায় ডেটা কম্প্রেশন (Data Compression) কেন প্রয়োজন, তা গাণিতিক হিসাব এবং উদাহরণের মাধ্যমে বোঝানো হয়েছে।

১. Raw Data-র সাইজ কীভাবে দ্রুত বৃদ্ধি পায় (টেবিল বিশ্লেষণ)

- **Grayscale Image (1024 × 1024):**
 - হিসাব: $1024 \times 1024 \times 8 \text{ bits} = 8,388,608 \text{ bits} \approx 1 \text{ MB}$
 - অর্থ: একটিমাত্র সাদাকালো ছবির র (Uncompressed) সাইজ দাঁড়ায় প্রায় 1 MB।
- **Full-Color RGB Frame (1920 × 1080):**
 - হিসাব: $1920 \times 1080 \times 24 \text{ bits} \approx 49,766,400 \text{ bits} \approx 5.93 \text{ MB}$
 - অর্থ: ১০৮০p রেজোলিউশনের একটিমাত্র রঙিন ছবির ফ্রেমের সাইজ প্রায় 5.93 MB।

- 30 fps Full HD Video:
 - হিসাব: $5.93 \text{ MB} \times 30 \text{ frames} \approx 178 \text{ MB/s}$
 - অর্থ: ১ সেকেন্ডের একটি ১০৮০p ভিডিও প্লে করতে 178 MB ডেটা লাগে! এটি সাধারণ স্টোরেজ বা ইন্টারনেটে স্ট্রিম করার জন্য অত্যন্ত বিশাল একটি সাইজ।
- CD-quality Audio (Stereo):
 - হিসাব: $44,100 \text{ samples/s} \times 16 \text{ bits} \times 2 \text{ channels} \approx 1.41 \text{ Mb/s}$
 - অর্থ: দীর্ঘ অডিও রেকর্ডিংয়ের ক্ষেত্রে এই র (Uncompressed) অডিও ফাইল অনেক জায়গা নেয়।

৩. Quick Check (একটি বাস্তব উদাহরণ)

যদি একটি 4MB-এর র (Raw) ছবি স্লো নেটওয়ার্কে পাঠাতে সমস্যা হয়, তবে সেটি কমপ্রেস করে 0.5MB-তে নামিয়ে আনলে কী সুবিধা হবে?

- উত্তর: কম ব্যান্ডউইথ লাগবে, দ্রুত ট্রান্সফার হবে এবং স্টোরেজ কম খরচ হবে।

কম্পিউটার সায়েন্স ও ডেটা নেটওয়ার্কিংয়ে 1 MB (মেগাবাইট)-কে সাধারণত দুটি ভিন্ন পদ্ধতিতে বিটে (bits) রূপান্তর করা হয়:

- স্ট্যান্ডার্ড / ডাটা ট্রান্সমিশন হিসাব (Base-10):

$$1 \text{ MB} = 1,000 \text{ KB} = 1,000,000 \text{ Bytes} = 1,000,000 \times 8 \text{ bits} = \mathbf{8,000,000 \text{ bits}}$$

- কম্পিউটার মেমোরি / মেগাবাইট হিসাব (Base-2, MiB / Binary):

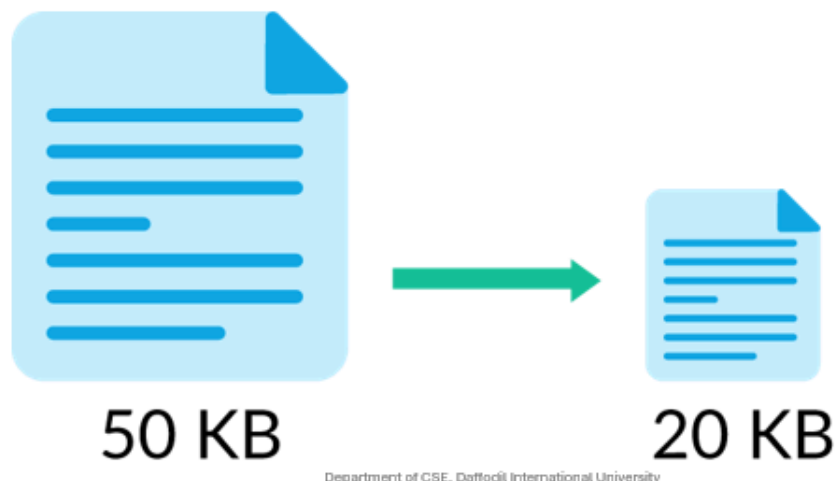
$$1 \text{ MB} = 1,024 \text{ KB} = 1,024 \times 1,024 \text{ Bytes} = 1,048,576 \text{ Bytes}$$

$$1,048,576 \times 8 \text{ bits} = \mathbf{8,388,608 \text{ bits}}$$

(তোমার আগের স্লাইডের মতো মাল্টিমিডিয়া মেমোরির অংকে সাধারণত Base-2 ব্যবহার করা হয়, তাই 1MB = 8,388,608 bits ধরা হয়েছে।)

2. Data Compression Basics

Before JPEG and MPEG, learn redundancy and measurement.



Data, Information and Redundancy

Data

Raw symbols or values stored in a computer. For an image, pixel intensities are data.

Information

Meaning extracted from the data. Different data representations may contain the same useful information.

Redundancy

Extra or repeated data that is not necessary for representing the same information.

Key idea: data is not always equal to information.

- If neighboring pixels are almost the same, storing each one independently wastes bits.
- If a symbol appears very often, giving every symbol the same bit length is inefficient.
- If the human eye cannot notice a tiny change, storing that detail exactly may be unnecessary in lossy compression.

Example

Consider 8 pixels:
120,120,121,120,120,121,120,120.
They carry nearly the same visual information. A smart coding method can represent this more compactly than storing each value independently.

৩. Data, Information, এবং Redundancy (রিডান্ডেন্সি)

টার্ম

সহজ সংজ্ঞা

Data (ডেটা)

কম্পিউটারে জমা থাকা raw symbols/মান — যেমন pixel-এর intensity value

Information (তথ্য)

ডেটা থেকে বের করা প্রকৃত অর্থ (meaning) — একই তথ্য বিভিন্ন representation-এ থাকতে পারে

Redundancy (অতিরিক্ততা)

ডেটার সেই অংশ যেটা বাড়তি/পুনরাবৃত্ত (extra/repeated), একই তথ্য বোঝাতে যেটার দরকার নেই

মূল কথা: Data সবসময় = Information না। যেমন — যদি পাশাপাশি (neighboring) pixel-গুলোর মান প্রায় একই হয়, তাহলে প্রতিটা আলাদাভাবে জমা রাখা অপচয় (wasteful)। **Compression** মূলত এই redundancy-টাই বের করে ফেলে দেয়।

Types of Redundancy



1. Coding redundancy

Some symbols occur much more often than others. Fixed-length code wastes bits.

Example: If A appears 70% of the time and Z appears 1% of the time, giving both 8 bits is inefficient. Variable-length coding such as Huffman helps here.

2. Spatial / temporal redundancy

Neighboring pixels in an image are often similar. Nearby video frames are also similar.

Example: blue sky across many pixels, or a person slightly moving between frames.

3. Psychovisual redundancy

Some details are less noticeable to the human eye. Lossy methods remove or coarsen them.

Example: slight high-frequency chroma detail in JPEG.

Question: Which redundancy is mainly reduced by Huffman coding? Which by MPEG motion compensation? Which by JPEG quantization?

১. Types of Redundancy (তিনটি ধরণ)

○ 1. Coding Redundancy:

- ব্যাখ্যা: কিছু চিহ্ন বা অ্যালফাবেট বারবার দেখা যায়, আবার কিছু খুব কম দেখা যায়। সবাইকে সমান সাইজের বিট (Fixed-length code) দিলে মেমোরি নষ্ট হয়।
- উদাহরণ: কোনো লেখায় 'A' যদি 70% সময় থাকে আর 'Z' যদি 1% সময় থাকে, তবে উভয়কে ৮ বিট করে দেওয়া অপচয়। এ ক্ষেত্রে Huffman Coding-এর মতো Variable-length coding ব্যবহার করে ঘনঘন আসা 'A'-কে ছোট বিট এবং কম আসা 'Z'-কে বড় বিট দেওয়া হয়।

○ 2. Spatial / Temporal Redundancy:

- Spatial (স্থানভিত্তিক): একটি ছবির পাশাপাশি থাকা পিক্সেলগুলো প্রায় একই রঙের হয় (যেমন — নীল আকাশের ছবি)।
- Temporal (সময়ভিত্তিক): ভিডিওর পাশাপাশি দুটি ফ্রেমের মাঝে বস্তু বা মানুষের অবস্থান সামান্যই বদলায়।
- উদাহরণ: ভিডিওতে কোনো মানুষের হাঁটার সময় ব্যাকগ্রাউন্ড অপরিবর্তিত থাকে, তাই প্রতি ফ্রেমে ব্যাকগ্রাউন্ড রি-ড্র না করে শুধু পরিবর্তিত অংশটুকু সেভ করা যায়।

- **3. Psychovisual Redundancy:**
 - ব্যাখ্যা: মানুষের চোখের দেখার সীমাবদ্ধতা রয়েছে। ছবির সূক্ষ্ম কিছু পরিবর্তন বা কালার ডিটেইলস চোখ ধরতে পারে না।
 - উদাহরণ: Lossy Compression (যেমন JPEG)-এ এমন কিছু কালার ডিটেইল বা হাই-ফ্রিকোয়েন্সি ডেটা বাদ দেওয়া হয়, যা বাদ দিলেও মানুষের চোখে পার্থক্য বোঝা যায় না।

২. Answer to Bottom Questions (নিচের প্রশ্নগুলোর উত্তর)

- **Question 1: Which redundancy is mainly reduced by Huffman coding?**
 - **Answer:** Coding redundancy.
- **Question 2: Which by MPEG motion compensation?**
 - **Answer:** Temporal redundancy (Video frames-এর জন্য).
- **Question 3: Which by JPEG quantization?**
 - **Answer:** Psychovisual redundancy.

Lossless vs Lossy Compression

Feature	Lossless	Lossy
Can reconstruct original exactly?	Yes	No
Main idea	Remove redundancy without losing information	Remove redundancy + discard less-important information
Typical methods	Huffman, Run-Length, PNG, ZIP	JPEG, MPEG, MP3
Typical use	Text, medical records, source code, archival	Photos, video, streaming, social media
Compression ratio	Usually lower	Usually higher

Remember

Lossless means exact recovery. Lossy means acceptable recovery based on perception or application requirement.

Question

Why is JPEG preferred for photographs but not for source code or legal documents?

8. Lossless বনাম Lossy Compression

এই স্লাইডটিতে **Lossless** (ক্ষতিহীন) এবং **Lossy** (ক্ষতিযুক্ত) কম্প্রেশনের মধ্যে পার্থক্য এবং মূল ধারণা সহজভাবে বোঝানো হয়েছে।

i) Lossless vs Lossy Compression (তুলনামূলক পার্থক্য)

বৈশিষ্ট্য (Feature)

Lossless (লসলেস)

Lossy (লসি)

মূল ডেটা হ্রাস ফেরত পাওয়া যায়?	হ্যাঁ (Yes); অরিজিনাল ডেটা 100% রিকভার করা যায়।	না (No); ডিপ্রেশনের পর অরিজিনাল ডেটার কিছু অংশ চিরতরে হারিয়ে যায়।
মূল ধারণা (Main Idea)	কোনো তথ্য না হারিয়ে কেবল অতিরিক্ত ডেটা (Redundancy) দূর করা হয়।	অপ্রয়োজনীয় ডেটা কমানোর পাশাপাশি মানুষের চোখে ধরা পড়ে না এমন তথ্য ফেলে দেওয়া হয়।
পদ্ধতি/ফরম্যাট (Methods)	Huffman, Run-Length, PNG, ZIP	JPEG, MPEG, MP3
ব্যবহার (Typical Use)	Text, Medical Records, Source Code, Archival Files	Photographs, Videos, Online Streaming, Social Media
কম্প্রেশন রেশিও (Compression Ratio)	সাধারণত কম (ফাইল সাইজ খুব বেশি ছোট হয় না)।	সাধারণত অনেক বেশি (ফাইল সাইজ অনেক ছোট করা যায়)।

ii) Remember (মূল বিষয়)

- **Lossless:** ডেটা হ্রাস পূর্বের অবস্থায় ফিরিয়ে আনা নিশ্চিত করে।
- **Lossy:** অ্যাপ্লিকেশনের প্রয়োজন অনুযায়ী গ্রহণযোগ্য মান বজায় রেখে ফাইল সাইজ ছোট করে।

iii) Answer to Question (নিচের প্রশ্নের উত্তর)

- **Question:** *Why is JPEG preferred for photographs but not for source code or legal documents?*
- **Answer:**
 - ফটো বা ছবির ক্ষেত্রে (**JPEG**): মানুষের চোখ ছবিতে কালারের সূক্ষ্ম পরিবর্তন বা সামান্য ডিটেইলসের ঘাটতি ধরতে পারে না। তাই Lossy Compression ব্যবহার করে গুণমান ঠিক রেখে ফাইল সাইজ অনেক ছোট করা সম্ভব হয়, যা ইন্টারনেটে দ্রুত লোড হতে সাহায্য করে।
 - সোর্স কোড বা আইনি নথির ক্ষেত্রে (**Source Code / Legal Documents**): এগুলোতে একটিমাত্র চরিত্র, সেমিকোলন (;) বা শব্দ বদলে গেলে সম্পূর্ণ প্রোগ্রামে এরর আসবে বা ডকুমেন্টের আইনি অর্থ পরিবর্তন হয়ে যাবে। তাই এখানে কোনো ডেটা হারানো গ্রহণযোগ্য নয়, ফলে Lossless পদ্ধতি ব্যবহার করা বাধ্যতামূলক।



Compression Ratio and Redundancy: Math

Formulas

$$\text{Compression ratio: } C = \frac{\text{Original size}}{\text{Compressed size}}$$

$$\text{Relative redundancy: } R = 1 - \frac{1}{C}$$

$$\text{Average Bit Length} = \frac{\text{Size in bits}}{\text{No. of pixels}}$$

Worked example

An image size is 16×16 pixels. After applying compression, the total size becomes 720 bits.

$$C = \frac{720}{16 \times 16} = 2.81 \approx 3$$

Therefore, the average number of bits required to represent each pixel is 3 bits/pixel.

Worked example

An image size is 65,536 bytes. After compression it becomes 6,554 bytes.

$$R = 1 - \frac{1}{10} = 0.9 \quad C = \frac{65,536}{6,554} \approx 10$$

So, we say the image has about 10:1 compression and 90% relative redundancy.

Practice problem

A raw image is 2.4 MB and the compressed image is 0.3 MB.

1) Find the compression ratio.

2) Find the relative redundancy.

Solution outline:

$$R = 1 - \frac{1}{8} = 0.875 = 87.5\% \quad C = \frac{2.4}{0.3} = 8$$

Department of CSE, Daffodil International University

১. Formulas (সূত্রসমূহ)

- Compression Ratio (C):

$$C = \frac{\text{Original Size}}{\text{Compressed Size}}$$

- Relative Redundancy (R):

$$R = 1 - \frac{1}{C}$$

- Average Bit Length:

$$\text{Average Bit Length} = \frac{\text{Size in bits}}{\text{No. of pixels}}$$

২. Slide Worked Examples (স্লাইডের উদাহরণ সমাধান)

- ডানদিকের ওপরের উদাহরণ:
 - দেওয়া আছে: পিক্সেল সংখ্যা = $16 \times 16 = 256$, কমপ্রেসড সাইজ = 720 bits।
 - $\text{Average Bit Length} = \frac{720}{16 \times 16} = \frac{720}{256} = 2.81 \approx 3 \text{ bits/pixel}$

- বামদিকের নিচের উদাহরণ:
 - দেওয়া আছে: অরিজিনাল সাইজ = 65, 536 bytes, কমপ্রেসড সাইজ = 6, 554 bytes।
 - Compression Ratio (C) = $\frac{65,536}{6,554} \approx 10$ (অর্থাৎ 10 : 1)।
 - Relative Redundancy (R) = $1 - \frac{1}{10} = 0.9 = 90\%$ ।

৩. Practice Problem Solution (কমপ্লিট সমাধান)

প্রশ্ন:

একটি র (Raw) ছবির সাইজ 2.4 MB এবং কমপ্রেস করার পর সাইজ হয় 0.3 MB।

1. Compression Ratio বের করো।
2. Relative Redundancy বের করো।

সমাধান Step-by-Step:

1. Compression Ratio (C):

$$C = \frac{\text{Original Size}}{\text{Compressed Size}} = \frac{2.4 \text{ MB}}{0.3 \text{ MB}} = 8$$

(অর্থাৎ কম্প্রেশন রেশিও 8 : 1)

2. Relative Redundancy (R):

$$R = 1 - \frac{1}{C} = 1 - \frac{1}{8} = 1 - 0.125 = 0.875$$

(শতাংশে প্রকাশ করলে: $0.875 \times 100\% = 87.5\%$)

উত্তর:

- Compression Ratio: 8 (বা 8 : 1)
- Relative Redundancy: 0.875 (বা 87.5%)

১. Compression Ratio (C): ফাইল কতগুণ ছোট হলো?

- সহজ অর্থ: কম্প্রেশন করার পর মূল ফাইল সাইজ কত গুণ কমলো, তা এই রেশিও নির্দেশ করে।
- বাস্তব উদাহরণ: $C = 8$ বা $8 : 1$ পাওয়ার অর্থ হলো মূল ফাইলটি আগের চেয়ে ৮ গুণ ছোট করা সম্ভব হয়েছে। অর্থাৎ, আগে যেখানে ৮টি বাইট লাগত, এখন সেখানে মাত্র ১টি বাইট লাগছে।

২. Relative Redundancy (R): মূল ফাইলের কত শতাংশ অপ্রয়োজনীয় ডেটা বাদ গেল?

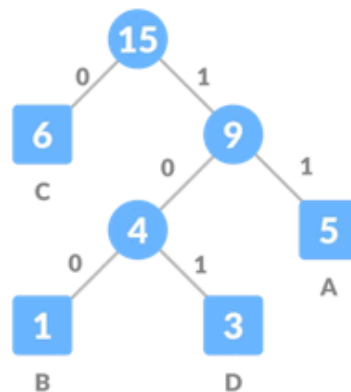
- সহজ অর্থ: অরিজিনাল ফাইলের ভেতরে কতটুকু অতিরিক্ত বা অপ্রয়োজনীয় ডেটা (Redundant Bits) ছিল যা কম্প্রেশনের সময় ছেঁটে ফেলা হয়েছে।
- বাস্তব উদাহরণ: $R = 0.875$ বা 87.5% পাওয়ার অর্থ হলো অরিজিনাল ফাইলের ৮৭.৫% অংশই মূলত অপ্রয়োজনীয় ডেটা ছিল, যা সফলভাবে কমিয়ে ফেলা সম্ভব হয়েছে।

৩. Average Bit Length: একটি পিক্সেলের পেছনে গড়ে কত বিট খরচ হচ্ছে?

- সহজ অর্থ: একটি ডিজিটাল ছবির প্রতিটি পিক্সেলকে মেমোরিতে ধরে রাখতে গড়ে কয়টি বিট প্রয়োজন পড়ছে।
- বাস্তব উদাহরণ: সাধারণ রঙিন ছবিতে প্রতি পিক্সেলের জন্য ২৪ বিট (24 bits/pixel) লাগে। কম্প্রেশন করার পর যদি Average Bit Length দাঁড়ায় 3 bits/pixel, তবে বোঝা যাবে প্রতি পিক্সেলে আমরা ২১ বিট করে মেমোরি সাশ্রয় করতে পেরেছি।

3. Huffman Coding

The classic lossless coding technique you should be able to solve by hand.



🎯 Part 2: Huffman Coding — সম্পূর্ণ ম্যাথ (Main Focus)

Huffman Coding: Main Idea

Definition

Huffman coding is a lossless variable-length coding technique. It assigns short codes to frequent pixels and longer codes to rare pixels.

Why it works

If common pixels use fewer bits, the average number of bits per symbol decreases. That means lower storage while preserving exact information.

Compare fixed vs variable length

Suppose four pixels use equal 2-bit fixed code. If one pixel appears much more often, variable-length coding can lower the average below 2 bits/pixel.

ধাপ ০: Huffman Coding কী ও কেন?

সংজ্ঞা: Huffman Coding হলো একটা **lossless, variable-length coding technique** (ভেরিয়েবল-দৈর্ঘ্যের কোডিং টেকনিক)। এটা বেশি ব্যবহৃত (**frequent**) pixel/symbol-কে ছোট কোড (**short code**) দেয়, আর কম ব্যবহৃত (**rare**) pixel/symbol-কে বড় কোড (**longer code**) দেয়।

কেন এটা কাজ করে? যদি common (বেশি আসা) pixel কম বিট (bit) ব্যবহার করে, তাহলে গড়ে (average) প্রতি pixel-এ যত বিট লাগে তা কমে যায় — ফলে storage কম লাগে, অথচ তথ্য একদম হুবহু (exact) থাকে (lossless)।

সহজ উদাহরণ দিয়ে ধারণা: ধরো ৪টা pixel value আছে, আর fixed-length code হলে প্রতিটাকে সমান ২-বিট (2-bit) কোড দিতে হবে (যেমন 00, 01, 10, 11)। কিন্তু যদি একটা value বার বার আসে (বাকিগুলোর চেয়ে বেশি), তাহলে variable-length coding দিয়ে (সেই বেশি-আসা value-কে ছোট কোড দিয়ে) গড় বিট সংখ্যা ২-এর নিচে নামানো যায়।

Bit এবং Byte-এর মূল সমীকরণ (Equations)

$$1 \text{ Byte (B)} = 8 \text{ Bits (b)}$$

$$1 \text{ Bit (b)} = \frac{1}{8} \text{ Byte} = 0.125 \text{ Byte}$$

মেমোরি ইউনিটের ধারাবাহিক হিসাব:

- 1 KB (Kilobyte) = 1,024 Bytes = $1,024 \times 8 = 8,192$ Bits
- 1 MB (Megabyte) = 1,024 KB = $1,024 \times 1,024 \text{ Bytes} = 8,388,608$ Bits

Pixel এবং Bit-এর মধ্যে সম্পর্ক

একটি পিক্সেল (Pixel) হলো ছবির ক্ষুদ্রতম অংশ, আর বিট (Bit) নির্ধারণ করে ওই পিক্সেলটির রঙ কত গভীর বা নিখুঁত হবে। কম্পিউটার গ্রাফিক্সের ভাষায় একে Color Depth বা Bit Depth বলা হয়।

- ১-বিট পিক্সেল (1-bit/pixel):
 - $2^1 = 2$ টি অবস্থা হতে পারে।
 - পিক্সেলটি কেবল ২টি রঙ দেখাতে পারবে (সাধারণত একদম সাদা অথবা কালো)।
- ৮-বিট পিক্সেল (8-bits/pixel):
 - $2^8 = 256$ টি ভিন্ন শেড বা অবস্থা হতে পারে।
 - এটি মূলত সাদাকালো বা Grayscale ছবিতে ব্যবহার করা হয় (০ থেকে ২৫৫ টি ধূসর শেড)।
- ২৪-বিট পিক্সেল (24-bits/pixel / True Color):
 - $2^{24} = 16,777,216$ টি (প্রায় ১.৬ কোটি) ভিন্ন রঙ তৈরি করতে পারে।
 - এখানে প্রতি পিক্সেলের Red (৮ বিট) + Green (৮ বিট) + Blue (৮ বিট) = মোট ২৪ বিট জায়গা লাগে।

সহজ সারসংক্ষেপ:

একটি ছবিতে যত বেশি পিক্সেল থাকবে, ছবি তত স্পষ্ট বা ডিটেইলড হবে (Resolution)। আর প্রতিটি পিক্সেলের জন্য যত বেশি বিট বরাদ্দ থাকবে, ছবিতে রঙের বৈচিত্র্য তত বেশি নিখুঁত হবে (Color Depth)।

"বার বারবার আসা" বলতে বোঝানো হয়েছে একটি ছবিতে কোনো নির্দিষ্ট Pixel Value (বা রঙের শেড) অন্যগুলোর তুলনায় অনেক বেশিবার রিইপিট হওয়া বা বারবার উপস্থিতি থাকা।

প্র্যাক্টিক্যাল একটি উদাহরণ দিয়ে বিষয়টি পানির মতো সহজ করে দিচ্ছি:

১. সিনারিও: নীল আকাশ ও একটা ছোট পাখি

ধরো একটি ছোট ইমেজে মোট ১০টি পিক্সেল আছে। ছবিতে নীল আকাশের অংশ বড়, তাই নীল রঙটা বারবার এসেছে।

১. সিনারিও: নীল আকাশ ও একটা ছোট পাখি

ধরো একটি ছোট ইমেজে মোট ১০টি পিক্সেল আছে। ছবিতে নীল আকাশের অংশ বড়, তাই নীল রঙটা বারবার এসেছে।

- পিক্সেল ভ্যালুগুলোর উপস্থিতি (Frequency):
 - A (গাঢ় নীল) → এসেছে ৭ বার
 - B (হালকা নীল) → এসেছে ১ বার
 - C (সাদা মেঘ) → এসেছে ১ বার
 - D (পাখির কালো রঙ) → এসেছে ১ বার

২. কেস ১: Fixed-Length Coding (সবাই সমান ২-বিট)

যেহেতু ৪টি আলাদা ভ্যালু (A, B, C, D), তাই সবাইকে প্রকাশ করতে সমান ২-বিট করে জায়গা দেওয়া হলো:

A = 00, B = 01, C = 10, D = 11

- মোট বিট খরচ:
১০টি পিক্সেলের জন্য $10 \times 2 = 20$ bits।
- গড় বিট (Average Bit):
 $\frac{20}{10} = 2$ bits/pixel।

৩. কেস ২: Variable-Length Coding (যে বারবার আসে তাকে ছোট কোড)

যেহেতু A রঙটি বারবার (৭ বার) এসেছে, তাই তাকে ছোট কোড দেওয়া হলো। আর যেগুলো ১ বার করে এসেছে, সেগুলোকে একটু বড় কোড দেওয়া হলো:

- A (৭ বার এসেছে) → কোড দেওয়া হলো ১-বিট (0)
- B (১ বার এসেছে) → কোড দেওয়া হলো ২-বিট (10)
- C (১ বার এসেছে) → কোড দেওয়া হলো ৩-বিট (110)
- D (১ বার এসেছে) → কোড দেওয়া হলো ৩-বিট (111)

- মোট বিট খরচ:
 - A -এর জন্য: $7 \times 1 = 7$ bits
 - B -এর জন্য: $1 \times 2 = 2$ bits
 - C -এর জন্য: $1 \times 3 = 3$ bits
 - D -এর জন্য: $1 \times 3 = 3$ bits
 - সর্বমোট: $7 + 2 + 3 + 3 = 15$ bits।
- গড় বিট (Average Bit):

$$\frac{15}{10} = 1.5 \text{ bits/pixel}$$

মূল সামারি

আগে যেখানে প্রতি পিক্সেলে গড়ে ২ বিট লাগছিল, ঘনঘন আসা A -কে ছোট কোড দেওয়ায় এখন গড় খরচ নেমে এসেছে ১.৫ বিটে!

কম্পিউটার গ্রাফিক্সের ভাষায় এটাই হলো Coding Redundancy কমানোর মূল ট্রিক (যা Huffman Coding-এ করা হয়)।

Huffman Coding Algorithm



Step-by-step algorithm

1. Count the frequency of each pixel value.
2. Put all pixel values into a min-priority queue.
3. Remove the two nodes with smallest frequency.
4. Combine them into a new parent node whose frequency is the sum.
5. Insert the parent back into the queue.
6. Repeat until one root remains.
7. Assign 0 to each left edge and 1 to each right edge.
8. The root-to-leaf path gives the code for each symbol.

Time-saving exam tip

In the exam, always write frequencies in descending order first. Then combine the two smallest repeatedly.

Important property

No Huffman code is a prefix of another Huffman code. Therefore, the bitstream can be decoded unambiguously.

Worked Example: Build a Huffman Tree

Given an image: 8*8 8-bit grayscale image

58	61	67	72	72	79	85	79
52	58	61	67	72	79	85	91
58	61	67	72	67	79	85	91
61	67	72	79	72	85	79	85
52	58	61	67	72	79	85	91
58	61	67	72	79	72	85	91
52	58	61	67	72	79	85	91
52	58	61	67	72	79	85	91

Pixel values Frequency

Gray level	Frequency
52	4
58	7
61	8
67	10
72	11
79	10
85	9
91	5

What to remember

Total Number of pixel in the image: $8 \times 8 = 64$. [Original Storage: (Total no. of pixel * bit) = $64 \times 8 = 512$ bits]

Sum of the frequency: $4+7+8+10+11+10+9+5=64$

Since, the total number of pixel and sum of the frequency are equal. Hence, the histogram is correct

Worked Example: Build a Huffman Tree

- Sort the frequency of pixel values in descending order to construct the Huffman tree
- Combine the two nodes with smallest frequency into a new parent node whose frequency is the sum
- Rearrange the nodes after making a new node

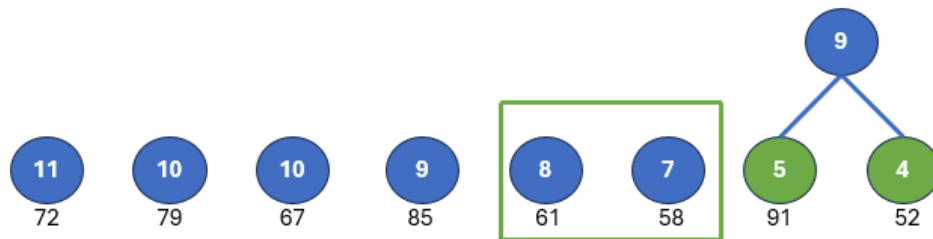


Note:

Since 4 and 5 are the two lowest frequencies, we combine them to form a new parent node. The value of the parent node is the sum of these two values: 9.

Worked Example: Build a Huffman Tree

- Sort the frequency of pixel values in descending order to construct the Huffman tree
- Combine the two nodes with smallest frequency into a new parent node whose frequency is the sum
- Rearrange the nodes after making a new node

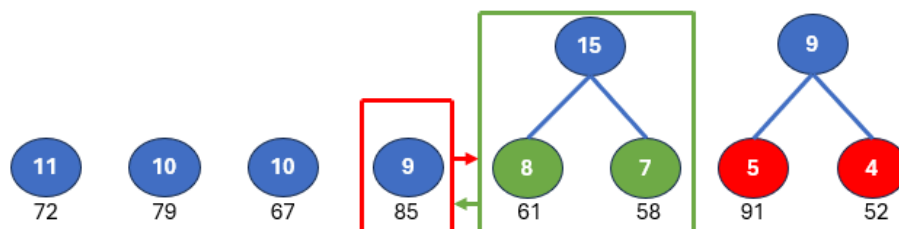


Note:

Now, lowest two nodes are 7 and 8. So, make a new parent nodes: $7+8=15$

Worked Example: Build a Huffman Tree

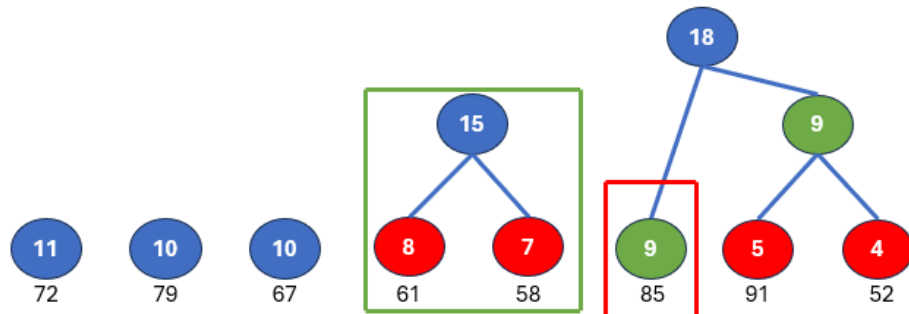
- Sort the frequency of pixel values in descending order to construct the Huffman tree
- Combine the two nodes with smallest frequency into a new parent node whose frequency is the sum
- Rearrange the nodes after making a new node



Note:

Since 9 and 15 are now the smallest nodes, we place them side by side for convenience so they can be merged into a new parent node.

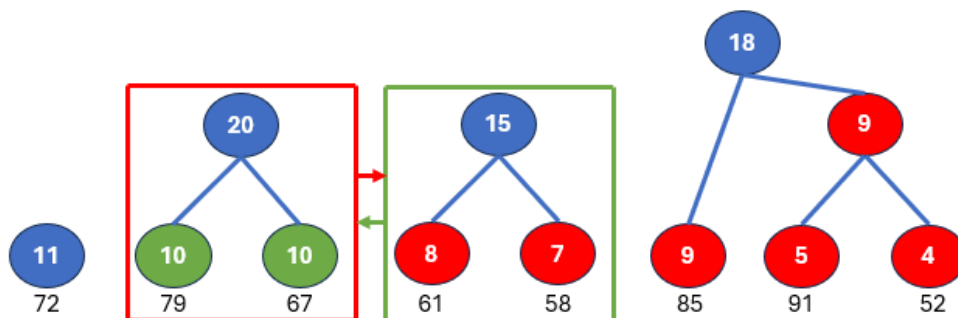
Worked Example: Build a Huffman Tree



Note:

Among 11, 10, 10, 15, and 18; node 10 and 10 are the smallest.

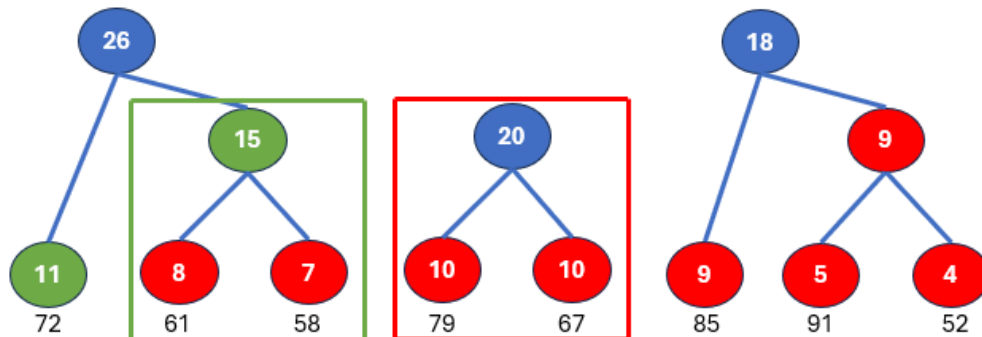
Worked Example: Build a Huffman Tree



Note:

Again, 11 and 15 are now the smallest nodes. So, we again place them side by side for convenience so they can be merged into a new parent node.

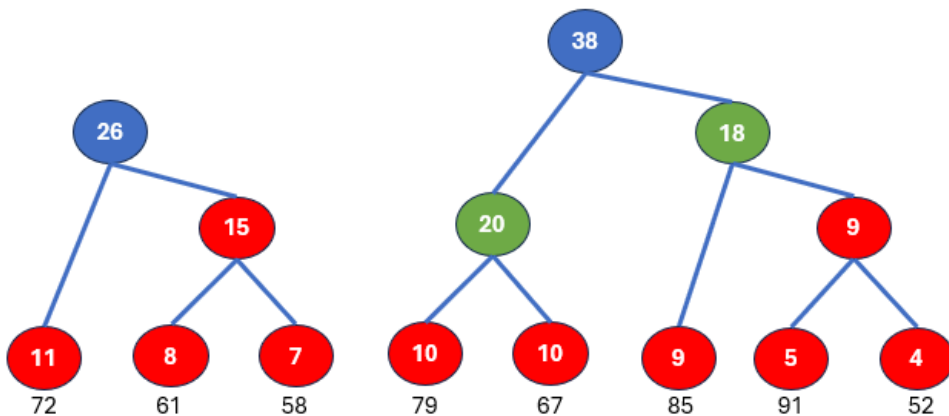
Worked Example: Build a Huffman Tree



Note:

Merge 18 and 20 as these two are now the smallest two nodes.

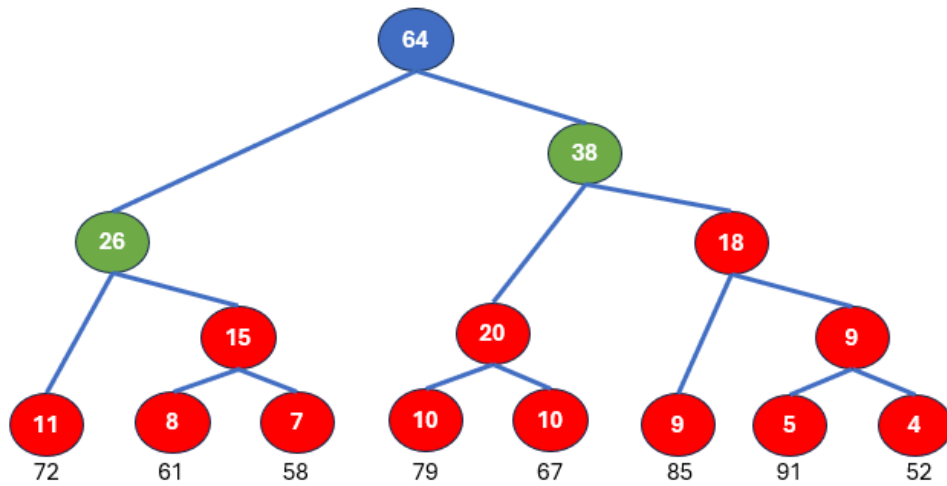
Worked Example: Build a Huffman Tree



Note:

Finally, merge 26 and 38 to form the root node, whose value is 64. This matches the total sum of all frequencies, which is 64. Since the image size is 8×8 , the total number of pixels is also 64.

Worked Example: Build a Huffman Tree

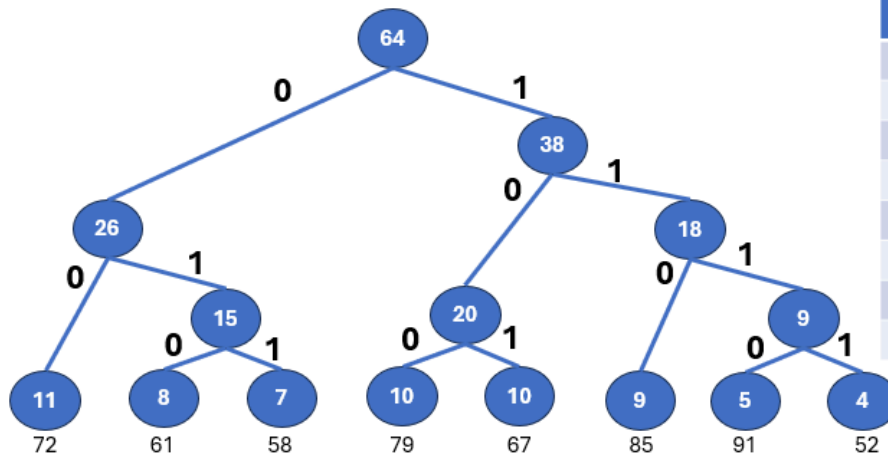


Note:

Now assign 0 to the left edge and 1 to the right edge.

Department of CSE, Daffodil International University

Worked Example: Build a Huffman Tree



Pixel Values	Codes	Bits	Freq.	Total (bit size x freq)
72	00	2	11	22
61	010	3	8	24
58	011	3	7	21
79	100	3	10	30
67	101	3	10	30
85	110	3	9	27
91	1110	4	5	20
52	1111	4	4	16

Note:

After applying Huffman coding, the size of the image becomes: $22+24+21+30+30+27+20+16 = 190$ bits

Department of CSE, Daffodil International University

Original vs compressed

Original size
 $= 64 \text{ pixels} \times 8 \text{ bits} = 512 \text{ bits}$

Compressed size = 190 bits

Compression ratio

Compression ratio,

$$C = \frac{\text{Original}}{\text{Compressed}} \\ = \frac{512 \text{ bits}}{190 \text{ bits}} \\ = 2.69 : 1$$

Relative redundancy,

$$R = 1 - \frac{1}{C} \\ = 1 - \frac{1}{2.69} = 0.372 \\ = 37.2\%$$

Bits per pixel and saving
 $= 2.97 \text{ bits/pixel}$

$\approx 3 \text{ bits/pixels}$

$$= \frac{190}{64} \text{ Average bits per pixel}$$

$\approx 3 \text{ bits/pixels}$

$$\text{Saving} = \frac{512 - 190}{512 \times 100} \\ = 62.89\%$$

Conclusion:

Huffman coding keeps the image information exact, but reduces the average storage from 8 bits/pixel to about 3 bits/pixel for this patch

What to remember:

The highest-frequency pixel value 72 gets the shortest code. Rare pixel values 52 and 91 get longer codes. That is why the average bit length decreases.

Practice Problem: Solve Huffman Coding by Hand

Problem

A 3x3 10-bit Grayscale image:

52	52	52	55
52	55	55	61
52	52	61	61
55	52	61	70

Tasks:

1. Draw the Huffman tree.
2. Write one valid code for each pixel.
3. Compute average bits/pixel.
4. Compare with fixed-length coding.

Answer outline

Possible combining order:

$$4+1=5$$

$$5+4=9$$

$$9+7=16 \text{ (Root)}$$

One valid code set:

$$52=0, 55=10, 61=110, 70=111$$

Lengths: 1,2,3,3

$$= \frac{30}{16} = 1.875 \text{ bits/pixel}$$

$$\approx 2 \text{ bits/pixel Avg bits/pixel}$$

$$= \frac{(1 \times 7) + (2 \times 4) + (3 \times 4) + (3 \times 1)}{16}$$

Fixed-length = 10 bit/pixel

Huffman Algorithm-এর ধাপ (Step-by-step)

1. প্রতিটা pixel value-এর **frequency** (কতবার এসেছে) গণনা করো
2. সব pixel value-কে একটা **min-priority queue**-তে রাখো (মানে frequency অনুযায়ী সাজানো)
3. সবচেয়ে ছোট দুইটা **frequency** নোড (node) সরিয়ে নাও
4. এই দুইটা নোডকে একসাথে করে একটা নতুন **parent node** বানাও, যার frequency = দুইটার যোগফল (sum)
5. এই parent node-কে আবার queue-তে ঢুকিয়ে দাও
6. এভাবে repeat করো, যতক্ষণ না একটাই **root** অবশিষ্ট থাকে
7. প্রতিটা **left edge**-কে 0, আর প্রতিটা **right edge**-কে 1 ধরো

8. **root** থেকে **leaf** (পাতা) পর্যন্ত পথ (**path**) ধরে যে 0/1 সিরিজ পাওয়া যায়, সেটাই ঐ **symbol**-এর **Huffman Code**

গুরুত্বপূর্ণ প্রপাটি (**exam-এ** লেখার মতো): কোনো Huffman code অন্য কোনো Huffman code-এর **prefix** (শুরুর অংশ) হয় না। এর মানে, bitstream (0,1-এর ধারা) থেকে কোনো ভুল বা সন্দেহ ছাড়াই (unambiguously) আসল symbol-গুলো ফিরে পাওয়া যায়।

এক্সাম টিপস: সবসময় frequency-গুলোকে প্রথমে **descending order** (বড় থেকে ছোট)-এ লিখে ফেলো, তারপর বারবার সবচেয়ে ছোট দুইটা মিলিয়ে যাও।

Worked Example: Build a Huffman Tree

Given an image: 8*8 8-bit grayscale image

58	61	67	72	72	79	85	79
52	58	61	67	72	79	85	91
58	61	67	72	67	79	85	91
61	67	72	79	72	85	79	85
52	58	61	67	72	79	85	91
58	61	67	72	79	72	85	91
52	58	61	67	72	79	85	91
52	58	61	67	72	79	85	91

Pixel values Frequency

Gray level	Frequency
52	4
58	7
61	8
67	10
72	11
79	10
85	9
91	5

What to remember

Total Number of pixel in the image: $8 \times 8 = 64$. [**Original Storage:** (Total no. of pixel * bit) = $64 \times 8 = 512$ bits]

Sum of the frequency: $4+7+8+10+11+10+9+5=64$

Since, the total number of pixel and sum of the frequency are equal. Hence, the histogram is correct

1 2 3 4 এখন সম্পূর্ণ **Worked Example** (তোমার স্লাইডের হুবহু সংখ্যা দিয়ে)

দেওয়া আছে (**Given**):

একটা **8×8, 8-bit grayscale image** (৮ বাই ৮ পিক্সেলের, ৮-বিট ধূসর মাত্রার ছবি):

```
58 61 67 72 72 79 85 79
52 58 61 67 72 79 85 91
58 61 67 72 67 79 85 91
61 67 72 79 72 85 79 85
52 58 61 67 72 79 85 91
58 61 67 72 79 72 85 91
52 58 61 67 72 79 85 91
52 58 61 67 72 79 85 91
```

ধাপ ১: সংশোধিত **Frequency Table** এবং যাচাই (**Verification**)

:

Gray Level	Frequency
52	4
58	7
61	8
67	9
72	11
79	10
85	9
91	6

যাচাই (Verification):

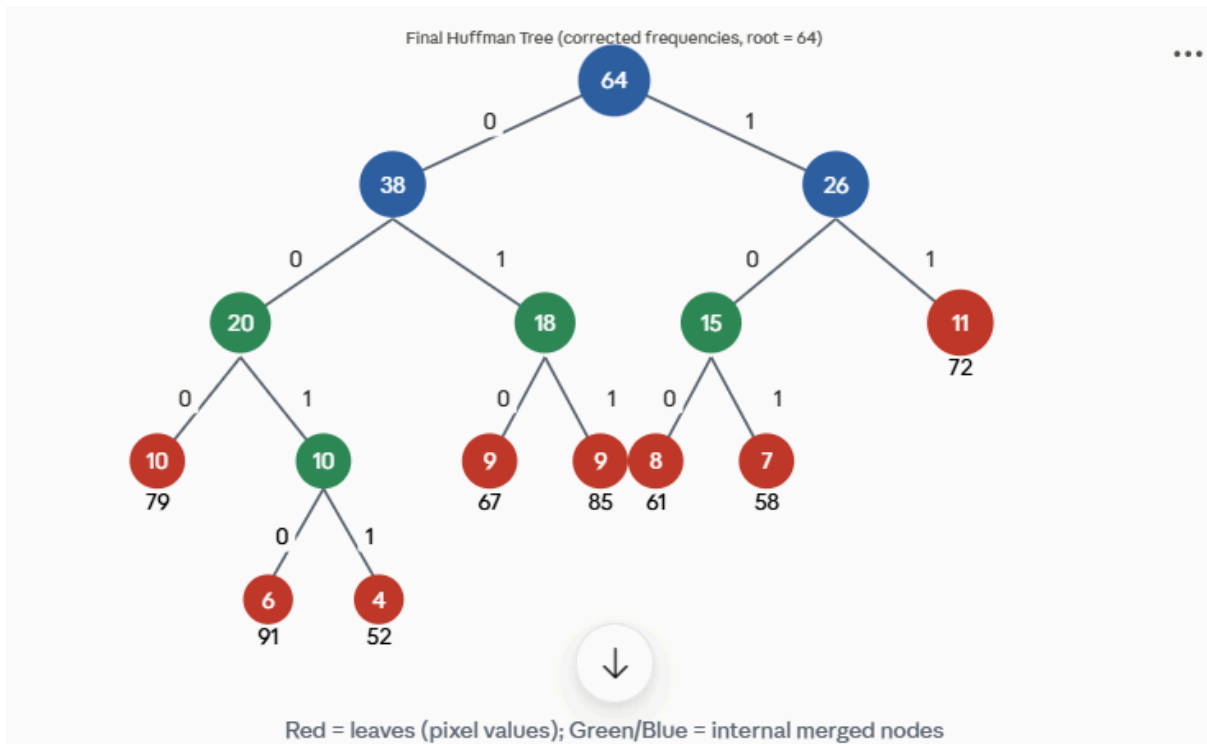
- মোট pixel = $8 \times 8 = 64$
- Frequency-এর যোগফল = $4+7+8+9+11+10+9+6 = 64$ ✓
- Original Storage = $64 \text{ pixel} \times 8 \text{ bit} = 512 \text{ bits}$ (এটা অপরিবর্তিত থাকবে, কারণ bit-depth একই)

ধাপ ২: **Huffman Tree** তৈরি করা (নতুন frequency দিয়ে)

Frequency descending order-এ সাজাই: **72:11, 79:10, 67:9, 85:9, 61:8, 58:7, 91:6, 52:4**

ধাপ	সবচেয়ে ছোট দুইটা মিলিয়ে	নতুন Node
1	$91(6) + 52(4)$	10 (N1)
2	$61(8) + 58(7)$	15 (N2)
3	$67(9) + 85(9)$	18 (N3)
4	$79(10) + N1(10)$	20 (N4)
5	$N2(15) + 72(11)$	26 (N5)
6	$N4(20) + N3(18)$	38 (N6)
7 (Root)	$N6(38) + N5(26)$	64

যাচাই: Root = 64 = মোট frequency-এর যোগফল  — Tree সঠিকভাবে তৈরি হয়েছে।



ধাপ ৩: **Code Assignment (Root থেকে Leaf পর্যন্ত পথ ধরে, Left=0, Right=1)**

Pixel Value	Path	Huffman Code	Code Length	Frequency
72	1-1	11	2	11
79	0-0-0	000	3	10
67	0-1-0	010	3	9
85	0-1-1	011	3	9
61	1-0-0	100	3	8
58	1-0-1	101	3	7
91	0-0-1-0	0010	4	6
52	0-0-1-1	0011	4	4

লক্ষ্য করো: এবার সবচেয়ে বেশি ব্যবহৃত **72 (frequency 11)** সবচেয়ে ছোট কোড (2 বিট) পাচ্ছে, আর সবচেয়ে কম ব্যবহৃত **52 (frequency 4)** সবচেয়ে বড় কোড (4 বিট) পাচ্ছে — একদম আগের নিয়মেই।

ধাপ ৪: **Compressed Size** হিসাব করা

Pixel	Bits	Freq	Total (Bits × Freq)
-------	------	------	---------------------

72	2	11	22
79	3	10	30
67	3	9	27
85	3	9	27
61	3	8	24
58	3	7	21
91	4	6	24
52	4	4	16

মোট **Compressed Size** = 22+30+27+27+24+21+24+16 = **191 bits**

ধাপ ৫: চূড়ান্ত হিসাব (Final Formulas)

① Original Size:

$$\text{Original Size} = 64 \times 8 = 512 \text{ bits}$$

② Compressed Size:

$$\text{Compressed Size} = 191 \text{ bits}$$

③ Average Bit Length (bits/pixel):

$$\text{Avg bits/pixel} = \frac{191}{64} = 2.984 \approx 3 \text{ bits/pixel}$$

④ Compression Ratio (C):

$$C = \frac{\text{Original}}{\text{Compressed}} = \frac{512}{191} = 2.68 : 1$$

⑤ Relative Redundancy (R):

$$R = 1 - \frac{1}{C} = 1 - \frac{191}{512} = 1 - 0.373 = 0.627 = 62.7\%$$

⑥ Savings:

$$\text{Saving} = 512 - 191 = 321 \text{ bits}$$

নতুন frequency অনুযায়ী Huffman coding ছবির তথ্য হুবহু অক্ষত রেখে গড় storage-কে 8 bits/pixel থেকে প্রায় 3 bits/pixel-এ নামিয়ে আনছে — কমপ্রেসন রেশিও 2.68:1, আর প্রায় 62.7% redundancy দূর করা সম্ভব হয়েছে।

⚠ একটা ছোট নোট (তোমার উপকারের জন্য): তোমার আপলোড করা স্লাইডের ছবিতে (আগের ভুল frequency দিয়ে) Relative Redundancy-এর হিসাবে একটা arithmetic slip ছিল — সেখানে লেখা ছিল " $R = 1 - 1/2.69 = 0.372$ ", কিন্তু সঠিকভাবে হিসাব করলে $1 - 1/2.69 = 0.628$ (62.8%) হওয়ার কথা, 0.372 না (0.372 আসলে শুধু $1/2.69$ -এর মান, "1 -" অংশটা বাদ পড়ে গেছে হিসাবে)। তাই এখানে আমি সঠিক ফর্মুলা প্রয়োগ করেই ($R = 1 - 1/C$) হিসাব করেছি। এক্ষেত্রে ফর্মুলা প্রয়োগ ঠিকভাবে দেখাতে পারলে এটাই সঠিক ও নিরাপদ পদ্ধতি।

🎯 দ্বিতীয় Practice Example (স্লাইড ২৬ থেকে — নিজে অনুশীলনের জন্য)

তোমার এক্ষেত্রে যদি অন্য সংখ্যা দিয়ে প্রশ্ন আসে, তাহলে ঠিক এই একই পদ্ধতি প্রয়োগ করবে। এখানে একটা ছোট উদাহরণ দিচ্ছি অনুশীলনের জন্য:

প্রশ্ন: একটা 3×3 ... আসলে স্লাইডে দেওয়া আছে **4x4, 10-bit grayscale image**:

```
52 52 52 55
52 55 55 61
52 52 61 61
55 52 61 70
```

Gray level ও frequency: 52 → 7 বার, 55 → 4 বার, 61 → 4 বার, 70 → 1 বার (মোট = 7+4+4+1 = 16 pixel ✓)

Merge অর্ডার:

- সবচেয়ে ছোট দুইটা: 4(70) + 1... (এখানে *actually smallest two frequencies* হলো 70:1 আর যেকোনো একটা 4, তাই) $4+1=5 \rightarrow 5+4=9 \rightarrow 9+7=16$ (Root)

একটা সম্ভাব্য বৈধ (**valid**) **code set**:

- 52 = 0 (length 1, কারণ এটা সবচেয়ে বেশি ব্যবহৃত)
- 55 = 10 (length 2)
- 61 = 110 (length 3)
- 70 = 111 (length 3)

Average bits/pixel:

$$Avg = \frac{(1 \times 7) + (2 \times 4) + (3 \times 4) + (3 \times 1)}{16} = \frac{7 + 8 + 12 + 3}{16} = \frac{30}{16} = 1.875 \approx 2 \text{ bits/pixel}$$

তুলনা: Fixed-length coding হলে লাগতো **10 bits/pixel** (কারণ এটা 10-bit image)! Huffman দিয়ে মাত্র ~2 bits/pixel-এ নামিয়ে আনা গেছে — বিশাল সঞ্চয় (savings)!

Exam-Ready ফর্মুলা শিট (সব একসাথে, মুখস্থ করো)

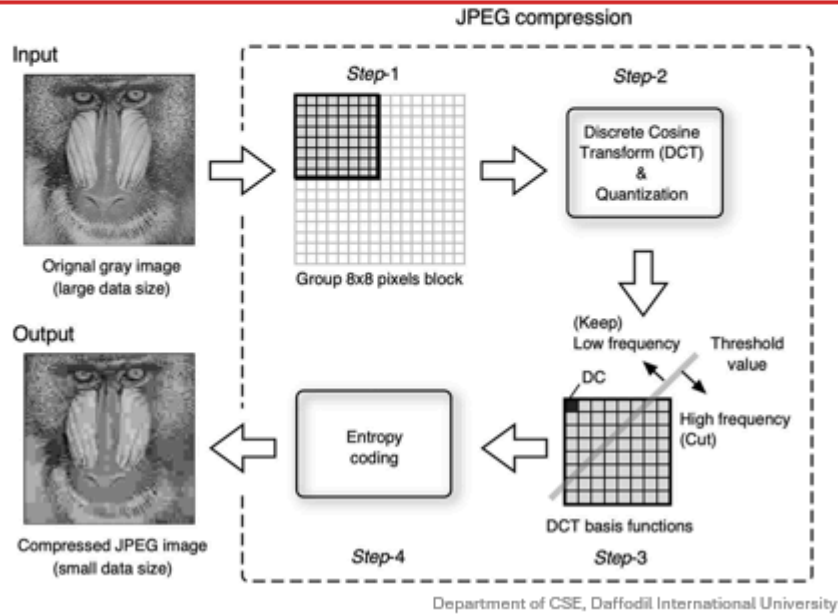
রাশি (Quantity)	ফর্মুলা
Original Size	Total pixels $\times$ bits/pixel (মূল bit depth)
Compressed Size	Σ (each symbol-এর code length $\times$ frequency)
Average Bit Length (bits/pixel)	Compressed Size $\div$ Total pixels
Compression Ratio (C)	Original Size $\div$ Compressed Size
Relative Redundancy (R)	$1 - \frac{1}{C}$
Savings	Original Size $-$ Compressed Size

এক্সামে ধাপে ধাপে যা যা লিখবে (**Answer** লেখার কাঠামো):

1. Matrix থেকে প্রতিটা gray level-এর frequency গুনে টেবিল বানাও, আর total pixel = sum of frequency তা যাচাই (verify) করে দেখাও।
2. Frequency descending order-এ সাজিয়ে ধাপে ধাপে Huffman Tree বানাও (প্রতি ধাপে কোন দুইটা merge হলো, তা লেখো)।
3. Root থেকে প্রতিটা leaf-এ গিয়ে 0/1 code বসিয়ে code table বানাও।
4. Compressed size = $\Sigma(\text{bits} \times \text{freq})$ বের করো।
5. চারটা ফাইনাল ফর্মুলা (Original size, Avg bit length, Compression ratio, Relative redundancy) প্রয়োগ করে answer লেখো।

4. JPEG Compression

Most common lossy still-image method. Learn the pipeline.



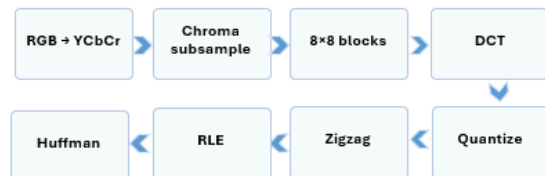
JPEG at a Glance

What JPEG is

JPEG is a lossy image compression standard designed mainly for natural photographs. It compresses by reducing psychovisual and some statistical redundancy.

Why JPEG works well for photos

Human vision is less sensitive to slight high-frequency and color-detail loss than to major brightness loss. JPEG uses this property to save many bits.



Remember the logic

Transform -> quantize -> reorder -> entropy-code. Quantization is the main lossy step.

5. JPEG Compression — সাধারণ ধারণা (JPEG at a Glance)

সংজ্ঞা: JPEG হলো একটা **lossy image compression standard** (ক্ষতিসাধনকারী ছবি কম্প্রেশন স্ট্যান্ডার্ড), যেটা মূলত **photograph** (ফটোগ্রাফ)-এর জন্য ডিজাইন করা হয়েছে। এটা ছবির **redundant high-frequency** (অপ্রয়োজনীয় উচ্চ-কম্পাঙ্ক) ও কালার ডিটেইল (**color detail**) কমিয়ে compress করে, আর কিছু **statistical redundancy** (পরিসংখ্যানগত অতিরিক্ততা) ব্যবহার করে।

কেন **JPEG** ছবির জন্য এত ভালো কাজ করে? মানুষের চোখ (human vision) হলো —

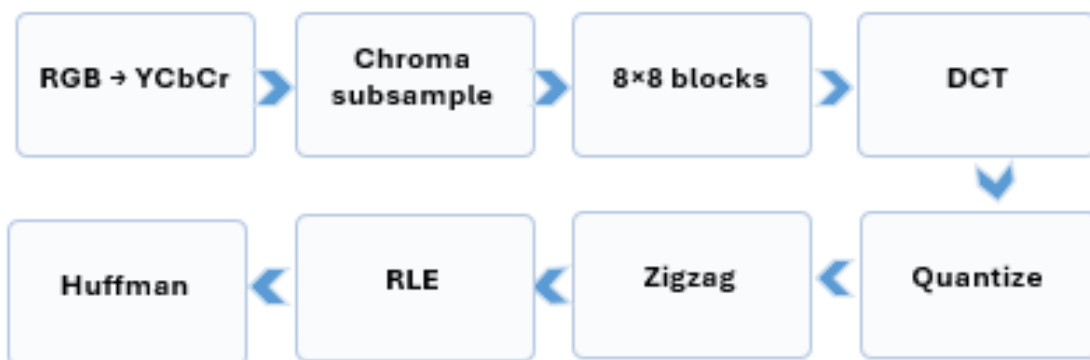
- উজ্জ্বলতার পরিবর্তনের (**brightness change**) প্রতি অনেক বেশি সংবেদনশীল
- কিন্তু উচ্চ-কম্পাঙ্ক ডিটেইল (**high-frequency detail**) এবং রঙের ক্ষুদ্র পরিবর্তনের (**color/chroma detail**) প্রতি ততটা সংবেদনশীল না

তাই JPEG এই দুর্বলতা (বরং সুবিধা!) কাজে লাগিয়ে সেই অংশগুলো compress করে ফেলে, যা চোখে তেমন ধরা পড়ে না — ফলে ফাইল সাইজ অনেক কমে, অথচ ছবি দেখতে প্রায় একই রকম লাগে।

Remember the logic

Transform -> quantize -> reorder -> entropy-code. Quantization is the main lossy step.

সংক্ষেপে যুক্তি (**Logic**) — মুখস্থ রাখো: **Transform** → **Reorder** → **Entropy-code**, যার মধ্যে **Quantization** (কোয়ান্টাইজেশন) ধাপটাই একমাত্র "lossy" (ক্ষতিসাধনকারী) ধাপ — বাকি সবকিছুই reversible (উল্টানো/ফেরানো সম্ভব)।



২.

JPEG-এর ধাপসমূহ (Steps 1-8, বিস্তারিত)

JPEG Step 1-3: Color Space and Blocking

1. RGB to YCbCr

Y represents brightness (luminance). Cb and Cr represent color difference (chrominance). JPEG often processes brightness more carefully than color.

2. Chroma subsampling

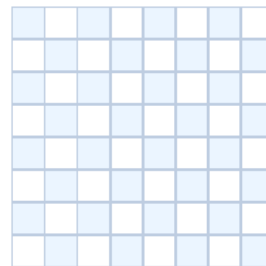
Because the eye is less sensitive to fine color detail, Cb and Cr may be stored at lower resolution, such as 4:2:0.

3. Divide into 8×8 blocks

The image is split into small 8×8 blocks. Each block is processed separately.

Advantage: localized computation.

Disadvantage: at very high compression, block boundaries may become visible.



Example 8×8 block

Quick concept check

Why can JPEG reduce more color detail than brightness detail? Because human vision notices luminance change more strongly.

Step 1: RGB to YCbCr (কালার স্পেস পরিবর্তন)

ছবির RGB representation-কে **YCbCr**-এ রূপান্তর করা হয়:

- **Y** = Luminance (উজ্জ্বলতা)
- **Cb, Cr** = Chrominance (রঙের তথ্য — নীলাভ ও লালচে উপাদান)

কেন এই রূপান্তর? কারণ Y (brightness), Cb ও Cr (color) থেকে আলাদা হয়ে গেলে, পরের ধাপে (chroma subsampling) সহজেই শুধু রঙের অংশটা কমপ্রেস করা যায়, brightness-কে অক্ষত রেখে।

Step 2: Chroma Subsampling (ক্রোমা সাব-স্যাম্পলিং)

মানুষের চোখ **fine color detail** (রঙের সূক্ষ্ম ডিটেইল)-এর চেয়ে **fine luminance detail** (উজ্জ্বলতার সূক্ষ্ম ডিটেইল)-এর প্রতি অনেক বেশি সংবেদনশীল। তাই Cb ও Cr-কে কম resolution-এ (যেমন **4:2:0**) সংরক্ষণ করা হয় — মানে প্রতি ৪টা luminance pixel-এর জন্য মাত্র ১টা করে Cb, Cr স্যাম্পল রাখা হয়। এতে অনেক জায়গা বাঁচে, কিন্তু চোখে তেমন পার্থক্য বোঝা যায় না।

Step 3: 8×8 Blocking (৮×৮ ব্লকে ভাগ করা)

পুরো ছবিকে ছোট ছোট **8×8 pixel**-এর ব্লকে (**block**) ভাগ করা হয়, আর প্রতিটা ব্লক আলাদাভাবে (**separately**) প্রসেস করা হয়।

- সুবিধা: Localized computation (স্থানীয়ভাবে হিসাব করা যায়, দ্রুত ও কার্যকর)
- অসুবিধা: খুব বেশি কমপ্রেসন করলে ব্লকের সীমানা (block boundary) মাঝে মাঝে দৃশ্যমান হয়ে যেতে পারে ("blocky" artifact)

Step 4: DCT — Discrete Cosine Transform (ডিসক্রিট কোসাইন ট্রান্সফর্ম)

সংজ্ঞা: DCT একটা mathematical transform, যেটা প্রতিটা 8×8 block-এর pixel value-কে **spatial domain** (স্থানিক ডোমেইন) থেকে **frequency domain** (কম্পাঙ্ক ডোমেইন)-এ রূপান্তর করে।

সহজ ভাষায় বুঝি: একটা ছবির ব্লকে DCT করলে সেটা একগুচ্ছ **frequency coefficient** (কম্পাঙ্ক-সহগ)-এ ভেঙে যায়:

- সবচেয়ে উপরে-বামে থাকা **coefficient** হলো **DC (Direct Current) coefficient** — এটা ব্লকের গড় **brightness (average brightness)** বোঝায়
- বাকি সব **coefficient**গুলো হলো **AC (Alternating Current) coefficient** — এগুলো ব্লকের ডিটেইল/পরিবর্তন (**detail/change**) বোঝায়

Low frequency vs High frequency: DCT-এর ফলাফলে, ব্লকের উপরের-বাম দিকে (**top-left**) থাকে **low-frequency** তথ্য (ছবির মূল, বড় আকৃতির তথ্য — যেমন সামগ্রিক আকৃতি, রং), আর নিচের-ডান দিকে (**bottom-right**) থাকে **high-frequency** তথ্য (সূক্ষ্ম, দ্রুত পরিবর্তনশীল ডিটেইল — যেমন টেক্সচার, শার্প কিনারা)।

Exam-এর জন্য মনে রাখার কথা: পুরো DCT ফর্মুলা মুখস্থ করে হাতে গণনা করা লাগবে না। শুধু জানতে হবে — DCT সবচেয়ে গুরুত্বপূর্ণ **visual** তথ্যকে অল্প কয়েকটা **low-frequency coefficient**-এ কেন্দ্রীভূত (**concentrate**) করে।

JPEG Step 5: Quantization (Main Lossy Stage)

Quantization formula:

$$Q_{coeff}(u, v) = \text{round}\left(\frac{F(u, v)}{Q(u, v)}\right)$$

Meaning

Large quantization values force many high-frequency coefficients toward zero. This removes fine detail and makes the later coding steps much more efficient.

This is why JPEG is lossy. Once coefficients are rounded, exact recovery is impossible.

Coefficient F	Q value	Rounded result
160	16	10
25	10	3
7	14	1
3	20	0
-9	17	-1

Mini exercise

Quantize 52 using Q=13.
Answer: round(52/13)=4

Quantize 6 using Q=20.
Answer: 0

Interpretation

After quantization, many small high-frequency values become zero. That makes zigzag + run-length coding very effective.

Step 5: Quantization (কোয়ান্টাইজেশন) — ⚠ এটাই একমাত্র Lossy ধাপ

ফর্মুলা:

$$Q_{ij}(u, v) = \text{round}\left(\frac{F(u, v)}{Q(u, v)}\right)$$

সহজ ভাষায়: প্রতিটা DCT coefficient (F)-কে একটা নির্দিষ্ট **quantization value (Q)** দিয়ে ভাগ করে, তারপর **round** (নিকটতম পূর্ণসংখ্যায় নিয়ে আসা) করা হয়।

উদাহরণ (স্লাইড ৩১ থেকে):

- Coefficient F = 52, Q = 13 $\rightarrow 52 \div 13 = 4.00 \rightarrow \text{round}(4) = 4$
- Coefficient F = 6, Q = 20 $\rightarrow 6 \div 20 = 0.3 \rightarrow \text{round}(0.3) = 0$

কেন এটা "lossy"? বড় quantization value (Q) দিয়ে ভাগ করলে অনেক ছোট, high-frequency coefficient রাউন্ড হয়ে 0-তে পরিণত হয় — আর একবার এই তথ্য 0 হয়ে গেলে, সেটা আর কখনোই আগের মতো ফিরে পাওয়া সম্ভব না (**exact recovery impossible**)। যেহেতু high-frequency detail-এর প্রতি মানুষের চোখ কম sensitive, তাই এই ক্ষতি চোখে তেমন ধরা পড়ে না, কিন্তু ডেটা সত্যিই হারিয়ে যায় — এই জন্যই JPEG-কে "lossy" বলা হয়।

JPEG Step 6-8: Zigzag, RLE and Huffman

Why zigzag?

Zigzag order reads low-frequency coefficients first and high-frequency coefficients later. After quantization, the later coefficients are often zero, producing long zero runs.

1	2	6	7
3	5	8	13
4	9	12	14
10	11	15	16

Simplified zigzag order

Run-Length Encoding example

Suppose zigzag output is:
[15, 3, 1, 0, 0, 0, -1, 0, 0, 0, 0]
RLE can store zeros compactly, e.g.,
(15),(3),(1),(0×3),(-1),(0×4)

Final entropy coding

The RLE symbols are then compressed again using Huffman coding. Therefore JPEG combines transform coding + lossy quantization + lossless entropy coding.

Step 6-8: Zigzag, RLE (Run-Length Encoding), এবং Huffman

Zigzag Scanning (জিগজ্যাগ স্ক্যানিং): Quantization-এর পর, 8×8 ব্লকের coefficient-গুলোকে একটা নির্দিষ্ট "zigzag" প্যাটার্নে সাজিয়ে একটা ১-মাত্রিক (1D) তালিকায় রূপান্তর করা হয়।

কেন zigzag-এই সাজানো হয়? কারণ এতে low-frequency coefficient (যেগুলো সাধারণত বড়/nonzero) একসাথে শুরুতে থাকে, আর high-frequency coefficient (যেগুলো quantization-এর পর প্রায়ই 0) একসাথে শেষের দিকে জমা হয় — ফলে লম্বা এক নাগাড়ে অনেকগুলো শূন্য (long run of zeros) তৈরি হয়।

RLE (Run-Length Encoding): একই মান বার বার (বিশেষ করে অনেকগুলো পরপর শূন্য - 0,0,0,0...) থাকলে, সেটাকে "কতবার এসেছে" এভাবে সংক্ষেপে লেখা হয় (যেমন ৫টা শূন্যকে "0×5" আকারে) — এতে অনেক জায়গা বাঁচে।

উদাহরণ: zigzag output = [15, 3, 1, 0, -1, 0, 0, 0, 0]-এর মতো একটা sequence-এ RLE প্রয়োগ করলে বার বার আসা শূন্যগুলোকে কমপ্যাক্ট আকারে লেখা যায় — যেমন (1,5)(3,1)(0,-1)-এই ধরনের (run, value) জোড়া আকারে।

সবশেষে **Huffman Coding**: RLE-এর আউটপুট symbol-গুলোকে আবার Huffman coding দিয়ে compress করা হয় (এটাই Part 2-তে আমরা বিস্তারিত শিখেছি!)।

তাই সম্পূর্ণ JPEG = Transform Coding (DCT) + Lossy Quantization + Lossless Entropy Coding (Zigzag+RLE+Huffman)।

৩. JPEG কেন ছবির জন্য ভালো (কিন্তু text/diagram-এর জন্য না)?

- **Photograph** (ছবি)-এর জন্য JPEG ভালো, কারণ ছবিতে সাধারণত smooth রঙের পরিবর্তন থাকে, আর সামান্য quality loss চোখে তেমন ধরা পড়ে না — কিন্তু ফাইল সাইজ অনেক কমে যায়।
- **Text** (লেখা) বা **sharp diagram** (স্পষ্ট রেখাযুক্ত ছবি)-এর জন্য JPEG ভালো না, কারণ quantization-এর ফলে অক্ষরের/রেখার তীক্ষ্ণ কিনারা (sharp edges) ঝাপসা (blurry) বা বিকৃত (artifact) হয়ে যেতে পারে — যেটা চোখে সহজেই ধরা পড়ে।

JPEG Worked Study Example

Problem

After DCT, assume a small coefficient sequence is:

[80, 18, 9, 3, 2, 1, 0, 0]

Quantization divisors are:

[8, 6, 9, 10, 12, 15, 20, 25]

Tasks:

1. Quantize each value.
2. Which values become zero?
3. Why does this help compression?

Solution

Quantized values:

$$80/8 = 10$$

$$18/6 = 3$$

$$9/9 = 1$$

$$3/10 \approx 0$$

$$2/12 \approx 0$$

$$1/15 \approx 0$$

$$0/20 = 0$$

$$0/25 = 0$$

Result: [10, 3, 1, 0, 0, 0, 0, 0]

Now the tail has a long run of zeros. Zigzag + RLE + Huffman can compress this very well.

২. Task 2: Which values become zero? (কোন মানগুলো শূন্য হয়ে গেল?)

- উত্তর: ছোট ছোট যে ভ্যালুগুলো ছিল— 3, 2, 1 (এবং শেষের 0, 0) কোয়ান্টাইজেশনের পর ভাগফল 0.5-এর নিচে হওয়ায় সবকটি 0 (শূন্য) হয়ে গেছে।

৩. Task 3: Why does this help compression? (এটি কীভাবে কম্প্রেশনে সাহায্য করে?)

- উত্তর: কোয়ান্টাইজেশনের পর সিকোয়েন্সের শেষে দীর্ঘ একটি শূন্যের সারি (Long run of zeros) তৈরি হয় ([10, 3, 1, 0, 0, 0, 0, 0])।
- এই পরপর থাকা শূন্যগুলোকে Zigzag Scanning, RLE (Run-Length Encoding) এবং Huffman Coding ব্যবহার করে খুব সহজেই অত্যন্ত কম বিটে প্রকাশ করা যায়।
- এতে ছবির অপ্রয়োজনীয় হাই-ফ্রিকোয়েন্সি ডিটেইলস (যা মানুষের চোখ ধরতে পারে না) বাদ পড়ে ফাইল সাইজ অনেক গুণ ছোট হয়ে যায়।

Practice Problems on JPEG

Problem 1

State the major steps of JPEG in correct order.

Expected order:
Color transform -> subsampling
-> blocking -> DCT ->
quantization -> zigzag -> RLE ->
Huffman

Problem 2

Which step of JPEG is lossy?
Why?

Expected answer:
Quantization. Because rounded
coefficient values cannot be
perfectly recovered.

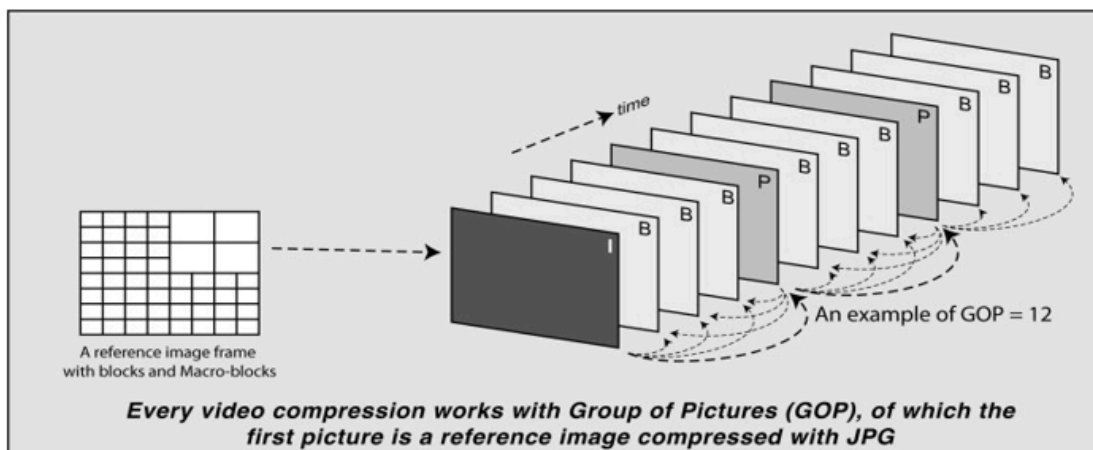
Problem 3

Why does JPEG work better
for photographs than for text
diagrams?

Expected idea:
Photos tolerate small smooth
losses better; sharp edges in
text/artifacts become more
visible.

5. MPEG Compression

From image compression to video compression.



From JPEG to MPEG

Core idea

A video is a sequence of frames. If we compressed every frame separately like JPEG, we would ignore the strong similarity between neighboring frames.

MPEG strategy

MPEG combines spatial compression inside each frame and temporal compression across frames. That is why it can achieve much higher video compression.

Real-world example

A talking person in front of a fixed background: most of the background remains the same from frame to frame. MPEG sends only the important change rather than sending the full frame each time.

8. MPEG Compression — ভিডিও কম্প্রেশন

JPEG থেকে MPEG-এ যাওয়া (From Image Compression to Video Compression)

মূল ধারণা: একটা ভিডিও (video) আসলে **frame**-এর একটা সিকোয়েন্স (**sequence of frames**) — অনেকগুলো স্থিরচিত্র (still image) একের পর এক দেখানো।

সমস্যা: যদি প্রতিটা frame-কে আলাদাভাবে **JPEG** দিয়ে **compress** করা হয় (independently), তাহলে পাশাপাশি থাকা (**neighboring**) frame-গুলোর মধ্যে যে বিশাল মিল (**similarity**) থাকে, সেটার কোনো সুবিধাই নেওয়া হবে না — যা একটা বিরাট অপচয় (waste)।

MPEG-এর কৌশল (strategy): MPEG প্রতিটা **frame**-এর ভেতরে (**within each frame**) — **spatial compression** (স্থানিক কম্প্রেশন) করে (ঠিক JPEG-এর মতো), আবার **frame**-থেকে-**frame** জুড়ে (**across frames**) — **temporal compression** (সময়গত/কালিক কম্প্রেশন)-ও করে। এই দুটো একসাথে মিলিয়ে MPEG অনেক বেশি (higher) video compression অর্জন করতে পারে।

উদাহরণ: একজন মানুষ কথা বলছে, ক্যামেরা স্থির (fixed background)। বেশিরভাগ background (পেছনের দৃশ্য) frame-থেকে-frame একই থাকে, শুধু মুখ ও হাতের সামান্য অংশ বদলায়। তাই MPEG পুরো frame আবার নতুন করে না পাঠিয়ে, শুধু যা পরিবর্তন হয়েছে (**only the important change**), সেটুকুই পাঠায় — এতে বিশাল compression পাওয়া যায়।

GOP — Group of Pictures (ছবির দল)

ভিডিওর প্রতিটা **compression** একটা **GOP (Group of Pictures)**-এর ভিত্তিতে কাজ করে। প্রতিটা GOP-এর প্রথম ছবিটাই (**first picture**) একটা **reference image** (রেফারেন্স ছবি), যেটা JPEG-এর মতো করে সম্পূর্ণভাবে compress করা থাকে — বাকি frame-গুলো এই reference-এর সাথে তুলনা করে (relative to it) তৈরি করা হয়।

MPEG Frame Types: I, P and B



Frame type	Meaning	Depends on	Use
I-frame	Intra-coded frame	No other frame	Reference, random access, highest size
P-frame	Predicted frame	Previous I/P frame	Stores motion + residual, medium size
B-frame	Bi-directional frame	Previous and next reference frames	Usually smallest, highest compression

Example GOP: I B B P B B P B B I

Meaning of GOP

GOP = Group of Pictures. It is the repeated pattern that organizes I, P and B frames.

Think

Why are I-frames larger? Because they are coded independently and must carry more complete image information.

MPEG-এর তিন ধরনের Frame (I, P, B) — খুবই গুরুত্বপূর্ণ

Frame Type	মানে	নির্ভর করে কার উপর	ব্যবহার/উদ্দেশ্য
I-frame	Intra-coded frame	অন্য কোনো frame-এর উপর না (কোনো নির্ভরতা নেই)	Reference frame, random access পয়েন্ট, সবচেয়ে ভালো quality কিন্তু সবচেয়ে বড় সাইজ
P-frame	Predicted frame	আগের P বা I-frame-এর উপর নির্ভরশীল	শুধু motion + residual (পার্থক্য) সংরক্ষণ করে, তাই ছোট সাইজ
B-frame	Bi-directional frame	আগের ও পরের দুই দিকের frame-এর উপর নির্ভরশীল	সবচেয়ে বেশি compression, কিন্তু encoding-এ complexity বেশি

GOP-এর উদাহরণ (স্লাইড ৩৭ অনুযায়ী): I B B P B P B P B B I → এখানে GOP শুরু হচ্ছে I দিয়ে, তারপর একগুচ্ছ B ও P ফ্রেম, তারপর আবার নতুন GOP শুরু হচ্ছে আরেকটা I দিয়ে। এই প্যাটার্নটাই বারবার (repeated pattern) ঘুরতে থাকে গোটা ভিডিও জুড়ে।

কেন **I-frame** বড়? কারণ I-frame কোনো অন্য frame-এর উপর নির্ভর করে না, তাই এতে সম্পূর্ণ ছবির তথ্য রাখতে হয় (JPEG-এর মতো)। এই কারণেই random access (মারপথ থেকে ভিডিও চালু করা, যেমন YouTube-এ কোনো নির্দিষ্ট সময়ে স্কিপ করা) I-frame থেকেই সম্ভব — কারণ এটা independently decodable (স্বাধীনভাবে ডিকোড করা যায়)।

Motion Compensation: The Heart of MPEG

Idea

Instead of sending the whole next block, MPEG searches for a similar block in a reference frame. It sends the motion vector and the residual difference.

Simple math example

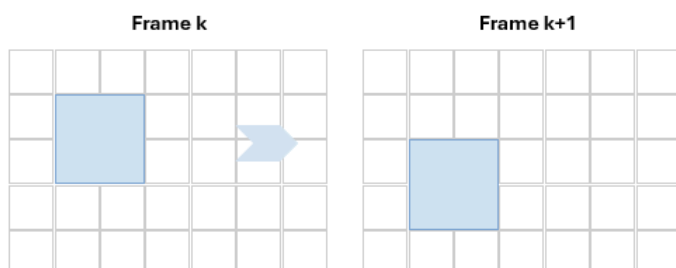
A block moves 2 pixels right and 1 pixel down.

Motion vector = (+2, +1)

Predicted block = block from old frame shifted by that vector.

Residual = actual new block - predicted block.

If residual is small, only a little extra data is needed.



Motion Compensation (মোশন কম্পেনসেশন) — MPEG-এর হৃদয় (The Heart of MPEG)

মূল আইডিয়া: পুরো নতুন block পাঠানোর বদলে, MPEG আগের reference frame-এ মিলে যাওয়া একটা **similar block** (একই রকম দেখতে ব্লক) খুঁজে বের করে (**searches**)। তারপর সেই ব্লকটার শুধু **motion vector** (গতির ভেক্টর/দিকনির্দেশ) আর **residual difference** (অবশিষ্ট পার্থক্য) পাঠায়।

সহজ গাণিতিক উদাহরণ (স্লাইড ৩৮): ধরো একটা ব্লক ডানে ২ পিক্সেল আর নিচে ১ পিক্সেল সরে গেছে (shift):

$$\text{Motionvector} = (+ 2, + 1)$$

- **Predicted block** = পুরনো (reference) frame থেকে সেই shift করা block
- **Residual** = আসল block – predicted block (এটা যদি খুব ছোট/near-zero হয়, তাহলে বাড়তি খুব সামান্য ডেটা পাঠালেই চলবে)

উদাহরণ দিয়ে সম্পূর্ণ বুঝি: ধরো একটা ভিডিওতে একটা বল (ball) স্ক্রিনে বাম থেকে ডানে নড়ছে, background স্থির। MPEG পুরো নতুন frame আবার না পাঠিয়ে বলে দেয়: "আগের frame-এর বলের block-টা শুধু ২ পিক্সেল ডানে সরে গেছে" (motion vector) — আর যদি এই prediction প্রায় নিখুঁত হয়, তাহলে residual (পার্থক্য) প্রায় শূন্যের কাছাকাছি, ফলে খুবই অল্প ডেটা পাঠালেই যথেষ্ট।

এই স্লাইডটিতে MPEG ভিডিও কম্প্রেশনের সবচেয়ে গুরুত্বপূর্ণ কৌশল Motion Compensation (মোশন ক্ষতিপূরণ/সমন্বয়) এর মূল ধারণা ও গাণিতিক নিয়ম সহজভাবে বোঝানো হয়েছে।

১. মূল ধারণা (The Main Idea)

পরপর দুটি ভিডিও ফ্রেমে (যেমন: Frame k এবং Frame k+1) একই অবজেক্ট বা ব্লক কেবল স্থান পরিবর্তন করে।

- MPEG পুরো নতুন ফ্রেমের ব্লকটিকে পুনরায় সেভ করে পাঠায় না।
- এর বদলে রেফারেন্স ফ্রেমে অবজেক্টটি কোথায় সরে গেছে তা খুঁজে বের করে এবং কেবল Motion Vector (গতির দিক ও দূরত্ব) এবং Residual Difference (পার্থক্য)টুকু পাঠায়।

২. Simple Math Example (সহজ গাণিতিক উদাহরণ)

ছবিতে দেখো:

- Frame k -তে একটি নীল ব্লক ছিল।
- Frame k+1 -এ ব্লকটি ডানদিকে ২ পিক্সেল (+2) এবং নিচের দিকে ১ পিক্সেল (+1) সরে গেছে।

ধাপসমূহ:

1. Motion Vector: কতটুকু সরল তার দিক নির্ণয় করা, Motion Vector = (+2, +1)।
2. Predicted Block: পুরোনো ব্লকটিকে মোশন ভেক্টর দিয়ে নতুন জায়গায় বসালে যে সম্ভাব্য ব্লক পাওয়া যায়।
3. Residual (অবশিষ্ট পার্থক্য): আসল নতুন ব্লক এবং প্রেডিক্টেড ব্লকের মধ্যে সামান্য পার্থক্যের হিসাব:

$$\text{Residual} = \text{Actual New Block} - \text{Predicted Block}$$

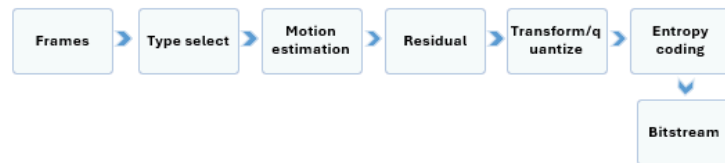
৩. কম্প্রেশনে এর সুবিধা

যদি অবজেক্টের রঙের বিশেষ পরিবর্তন না ঘটে, তবে Residual-এর মান প্রায় ০ (শূন্য) বা অত্যন্ত ক্ষুদ্র হয়। ফলে নতুন ফ্রেমের শত শত পিক্সেল সেভ না করে কেবল একটি মোশন ভেক্টর (+2, +1) এবং সামান্য রেসিডুয়াল পাঠালেই কাজ হয়ে যায়, যা ভিডিও ফাইলকে বহুগুণ ছোট করে ফেলে!

MPEG Encoding Pipeline

Pipeline overview

1. Divide video into frames.
2. Decide frame type: I, P or B.
3. For P/B frames, search matching blocks in reference frame(s).
4. Compute motion vectors.
5. Compute residual error.
6. Transform and quantize residual.
7. Entropy-code the result.



Key comparison with JPEG

JPEG works mainly within one image. MPEG also uses relationships between frames, so it removes temporal redundancy in addition to spatial redundancy.

MPEG Encoding Pipeline (সংক্ষেপে ৭টা ধাপ)

1. Frame-গুলোকে ভাগ করা (Divide frames)
2. প্রতিটার Frame Type (I, P, বা B) ঠিক করা
3. Matching block খুঁজে motion vector বের করা (P/B frame-এর জন্য)
4. Motion vector গণনা করা
5. Residual (পার্থক্য) গণনা করা
6. Residual-কে Transform + Quantize করা (অনেকটা JPEG-এর মতোই DCT+Quantization)
7. ফলাফল Entropy-code (Huffman-জাতীয়) করা → **Bitstream**

JPEG-এর সাথে মূল পার্থক্য (Key Comparison): JPEG শুধু একটা মাত্র ছবি নিয়ে কাজ করে, শুধু spatial redundancy দূর করে। MPEG একাধিক frame-এর মধ্যে সম্পর্ক (relationship) ব্যবহার করে, তাই spatial redundancy-এর পাশাপাশি temporal redundancy-ও দূর করে — এই কারণেই ভিডিওতে ছবির চেয়ে অনেক বেশি (higher) কম্প্রেশন সম্ভব হয়।

Practice Problems on MPEG

Problem 1

Why is MPEG usually better than compressing each frame separately with JPEG?

Expected answer:
Because neighboring frames are similar. MPEG exploits temporal redundancy using prediction and motion compensation.

Problem 2

A block moves from position (10,12) in one frame to (13,14) in the next.
Find the motion vector.

Answer: (+3, +2)

Problem 3

What is the role of an I-frame in a GOP?

Expected answer:
Reference frame, random access point, independently decodable frame.

One-Slide Summary

- Multimedia system = multiple media types + timing + interaction + synchronization.
- Compression reduces data while trying to keep useful information.
- Main redundancy types: coding, spatial/temporal, psychovisual.
- Lossless = exact recovery. Lossy = approximate recovery with higher compression.
- Huffman coding removes coding redundancy using variable-length codes.
- JPEG compresses still images by DCT + quantization + entropy coding.
- MPEG compresses video by combining image compression with motion-based prediction.

Last-minute revision tip

If you can explain why JPEG is good for images and MPEG is good for video, you already understand the central idea of this lecture.

Department of CSE, Cardiff International University

Mixed Practice Set

Theory questions

1. Define multimedia system with two examples.
2. Distinguish between data and information.
3. What is redundancy? Name three types.
4. Compare lossless and lossy compression.
5. Why is Huffman coding lossless?
6. Which step of JPEG is lossy and why?
7. Why does MPEG need I, P and B frames?

Numerical / algorithmic questions

1. A 1.2 MB file becomes 0.2 MB. Find C and R.
2. Build a Huffman code for a given frequency table.
3. Quantize a given DCT coefficient using a given Q value.
4. Find a motion vector from two block positions.
5. Explain how long zero runs appear in JPEG after quantization.

Theory Questions — উত্তরমালা

১. Define multimedia system with two examples.

উত্তর: Multimedia System হলো একটা **computer-based system**, যেটা একাধিক ধরনের মিডিয়া — যেমন **text, graphics, image, audio, video** — একসাথে **capture, store, compress, transmit** এবং **present** করে।

দুইটা উদাহরণ:

1. **YouTube** — ভিডিও + অডিও + সাবটাইটেল + থাম্বনেইল একসাথে দেখায়
2. **Online class platform** — স্লাইড + মাইক্রোফোনের অডিও + স্ক্রিন শেয়ার একসাথে চলে

২. Distinguish between data and information.

	Data	Information
সংজ্ঞা	কম্পিউটারে জমা থাকা raw symbols/মান	Data থেকে বের করা প্রকৃত অর্থ (meaning)
উদাহরণ	পিক্সেলের intensity মান — 120,120,121,120	ঐ মানগুলো বোঝাচ্ছে একটা সমতল, ধূসর অঞ্চল
মূল পার্থক্য	Data সবসময় Information না — data-তে redundancy থাকতে পারে যা extra info দেয় না	Information হলো data-র সারমর্ম, যা কমপ্যাক্টভাবেও প্রকাশ করা যায়

৩. What is redundancy? Name three types.

সংজ্ঞা: Redundancy হলো ডেটার সেই অংশ, যেটা বাড়তি/পুনরাবৃত্ত (extra/repeated), যা প্রকৃত তথ্য বোঝাতে একান্ত দরকার নেই — compression মূলত এই অংশটাই বাদ দেয়।

তিন ধরনের **redundancy**:

1. **Coding Redundancy** — fixed-length code ব্যবহারে জায়গার অপচয় (Huffman coding এটা সমাধান করে)
2. **Spatial/Temporal Redundancy** — পাশাপাশি pixel বা কাছাকাছি video frame প্রায় একই রকম হওয়া
3. **Psychovisual Redundancy** — মানুষের চোখ কিছু সূক্ষ্ম ডিটেইল খেয়ালই করে না (JPEG এটা কাজে লাগায়)

৪. Compare lossless and lossy compression.

বৈশিষ্ট্য	Lossless	Lossy
মূল ডেটা ফেরত পাওয়া যায়?	হ্যাঁ, হুবহু (exact)	না, আংশিক (approximate)
কী বাদ দেয়?	শুধু coding redundancy	Redundancy + কিছু কম-গুরুত্বপূর্ণ তথ্য
উদাহরণ	Huffman, PNG, ZIP	JPEG, MPEG, MP3
Compression ratio	তুলনামূলক কম	তুলনামূলক বেশি
ব্যবহার	টেক্সট, মেডিকেল ডেটা, সোর্স কোড	ছবি, ভিডিও, স্ট্রিমিং মিডিয়া

৫. Why is Huffman coding lossless?

উত্তর: Huffman coding শুধু **variable-length code** ব্যবহার করে symbol-গুলোকে represent করে — কোনো তথ্য বাদ দেয় না বা approximate করে না। প্রতিটা code অন্য কোনো code-এর **prefix** হয় না (prefix-free property), তাই bitstream থেকে কোনো **ambiguity** (দ্বিধা) ছাড়াই মূল ডেটা হুবহু (**exact**) ফেরত পাওয়া যায় — এই কারণেই এটা lossless।

৬. Which step of JPEG is lossy and why?

উত্তর: JPEG-এর **Quantization** ধাপটাই একমাত্র lossy ধাপ। কারণ, এই ধাপে প্রতিটা DCT coefficient-কে একটা Q-value দিয়ে ভাগ করে **round** (নিকটতম পূর্ণসংখ্যায়) করা হয় — এই রাউন্ডিং-এর ফলে অনেক ছোট, high-frequency coefficient 0-তে পরিণত হয়ে যায়, আর একবার এই তথ্য হারিয়ে গেলে তা আর কখনো হুবহু ফেরত পাওয়া সম্ভব না। বাকি সব ধাপ (color transform, DCT, zigzag, RLE, Huffman) সম্পূর্ণ reversible।

৭. Why does MPEG need I, P and B frames?

উত্তর: MPEG-এ ভিডিওর একটানা অনেক frame থাকে, যেগুলোর মধ্যে বিশাল মিল (temporal redundancy) থাকে। তাই —

- **I-frame** দরকার একটা স্বাধীন **reference point** হিসেবে (random access-এর জন্য, যেমন ভিডিওতে skip করা)
- **P-frame** দরকার আগের frame-এর সাথে তুলনা করে শুধু পরিবর্তনটুকু পাঠিয়ে জায়গা বাঁচাতে
- **B-frame** দরকার সামনে-পেছনে দুই দিকের frame ব্যবহার করে সর্বোচ্চ **compression** পেতে

এই তিন ধরনের frame মিলিয়ে ব্যবহার করলে ভিডিওতে **spatial + temporal** — দুই ধরনের redundancy-ই সর্বোচ্চ মাত্রায় দূর করা যায়, ফলে অনেক বেশি compression পাওয়া যায়, single-frame-ভিত্তিক JPEG-এর তুলনায়।

12 34 Numerical / Algorithmic Questions — সমাধান

১. A 1.2 MB file becomes 0.2 MB. Find C and R.

সমাধান:

$$C = \frac{\text{Original}}{\text{Compressed}} = \frac{1.2\text{MB}}{0.2\text{MB}} = 6:1$$

$$R = 1 - \frac{1}{C} = 1 - \frac{1}{6} = 1 - 0.1667 = 0.8333 = 83.33\%$$

উত্তর: Compression Ratio = **6:1**, Relative Redundancy = **83.33%**

২. Build a Huffman code for a given frequency table (Sample উদাহরণ দিয়ে পদ্ধতি)

ধরো একটা ছোট frequency table দেওয়া আছে: **A:5, B:9, C:12, D:13, E:16, F:45**

ধাপ: সবসময় দুইটা সবচেয়ে ছোট merge করে যাও:

ধাপ	Merge	নতুন Node
1	A(5)+B(9)	14
2	C(12)+D(13)	25
3	node14(14)+E(16)	30
4	node25(25)+node30(30)	55
5 (Root)	node55(55)+F(45)	100

Code বসানো (root→leaf, left=0, right=1):

- F = **0** (1 bit) — সবচেয়ে বেশি frequency, তাই সবচেয়ে ছোট code
- C = **100** (3 bit)
- D = **101** (3 bit)
- A = **1100** (4 bit)
- B = **1101** (4 bit)
- E = **111** (3 bit)

এভাবেই যেকোনো **frequency table** দেওয়া থাকলে — descending order-এ সাজাও, বারবার দুইটা ছোট merge করো, শেষে root থেকে path ধরে code বসাও।

৩. Quantize a given DCT coefficient using a given Q value (Sample উদাহরণ)

ফর্মুলা:

$$Q_{ij}(u, v) = \text{round}\left(\frac{F(u, v)}{Q(u, v)}\right)$$

উদাহরণ: ধরো DCT coefficient **F = 50**, আর Quantization value **Q = 16**।

$$Q_{ij} = \text{round}\left(\frac{50}{16}\right) = \text{round}(3.125) = 3$$

আরেকটা উদাহরণ: $F = 8$, $Q = 20$ হলে:

$$Q_{ij} = \text{round}\left(\frac{8}{20}\right) = \text{round}(0.4) = 0$$

লক্ষ্য করো: যখন coefficient (F) ছোট আর Q -value বড়, ফলাফল সহজেই **0** হয়ে যায় — এটাই high-frequency coefficient-গুলো শূন্যে পরিণত হওয়ার কারণ (এটাই পরের প্রশ্নের উত্তরের সাথে সরাসরি সম্পর্কিত)।

8. Find a motion vector from two block positions (Sample উদাহরণ)

ধরা যাক:

- Reference frame-এ block-টার অবস্থান (position) = ($x=10$, $y=12$)
- Current frame-এ একই (matching) block-টার অবস্থান = ($x=13$, $y=14$)

Motion Vector গণনা:

$$\text{MotionVector} = (x_{\text{current}} - x_{\text{ref}}, y_{\text{current}} - y_{\text{ref}})$$

$$= (13 - 10, 14 - 12) = (+3, +2)$$

অর্থ (**Interpretation**): ব্লকটা ডানদিকে ৩ পিক্সেল এবং নিচের দিকে ২ পিক্সেল সরে গেছে reference frame থেকে current frame-এ আসতে। এই motion vector-টাই P/B-frame-এ পাঠানো হয় (পুরো ব্লক আবার না পাঠিয়ে), সাথে **residual (actual block – predicted block)** — যা সাধারণত খুব ছোট হয়, তাই বিশাল জায়গা বাঁচে।

৫. Explain how long zero runs appear in JPEG after quantization.

উত্তর (ব্যাখ্যা):

১. **DCT**-এর পর একটা 8×8 ব্লকে low-frequency coefficient (উপরের-বাম দিকে) বড় মান পায়, আর high-frequency coefficient (নিচের-ডান দিকে) সাধারণত ছোট মান পায় — কারণ ছবির বেশিরভাগ শক্তি (energy) low-frequency অংশে জমা থাকে।

২. **Quantization**-এর সময়, ছোট high-frequency coefficient-গুলোকে বড় Q-value দিয়ে ভাগ করে round করা হয় — ফলে এই মানগুলোর বেশিরভাগই 0-তে পরিণত হয়ে যায় (যেমন উপরের প্রশ্ন ৩-এর উদাহরণে $F=8$, $Q=20$ হলে ফলাফল 0)।

৩. যেহেতু 8×8 ব্লকের নিচের-ডান কোণায় (high-frequency অঞ্চলে) বেশিরভাগ coefficient শূন্য হয়ে যায়, তাই সেই ব্লকটাকে যখন **Zigzag Scan** দিয়ে ১-মাত্রিক (1D) তালিকায় সাজানো হয় — তখন **low-frequency (nonzero)** মানগুলো শুরুতে, আর **high-frequency (zero)** মানগুলো একসাথে শেষের দিকে জমা হয়ে যায়।

৪. এর ফলে zigzag output-এর শেষ দিকে অনেকগুলো পরপর শূন্য (**long run of zeros**) তৈরি হয় — যেমন:
[23, -5, 3, 0, 1, 0, 0, 0, 0, 0, 0, ...]

৫. এই লম্বা শূন্যের ধারাকে **Run-Length Encoding (RLE)** দিয়ে খুব কমপ্যাক্টভাবে প্রকাশ করা যায় (যেমন "৭টা শূন্য" -কে এক জোড়া সংখ্যা দিয়েই বোঝানো যায়, ৭ বার আলাদাভাবে 0 না লিখে) — এটাই JPEG-এ বিশাল compression achieve করার একটা মূল কারণ।

সংক্ষেপে: Quantization → ছোট high-frequency coefficient শূন্যে পরিণত হয় → Zigzag scan সব শূন্যকে একসাথে জড়ো করে → লম্বা zero run তৈরি হয় → RLE সেটা কমপ্যাক্টভাবে এনকোড করে।

পুরো লেকচারের **One-Slide Summary** (স্লাইড ৪২ অনুযায়ী)

- **Multimedia system** = একাধিক মিডিয়া টাইপ + timing + interaction + synchronization
- **Compression** ডেটা কমায়, কিন্তু useful তথ্য যতটা সম্ভব রাখার চেষ্টা করে
- প্রধান **redundancy** ধরন: Coding, Spatial/Temporal, Psychovisual
- **Huffman coding** variable-length code দিয়ে **coding redundancy** দূর করে
- **JPEG** স্থির ছবি compress করে **DCT + Quantization + Entropy coding** দিয়ে
- **MPEG** motion prediction-এর সাথে image compression মিলিয়ে ভিডিও compress করে

শেষ মুহূর্তের রিভিশন টিপ (**Last-minute revision tip**): যদি তুমি ব্যাখ্যা করতে পারো "কেন **JPEG** ছবির জন্য ভালো, আর **MPEG** ভিডিওর জন্য ভালো" — তাহলে বুঝতে হবে এই পুরো লেকচারের মূল ধারণা তোমার একদম পরিষ্কার হয়ে গেছে।
